Personal Health Activity Tracker

AAI-530 (sp25) Group 5 Final Project

Robair Farag, Michael De Leon, Geoffrey Fadera

University of San Diego, Applied Artificial Intelligence

Date: February 24, 2025

GitHub Repository: https://github.com/Robairfarag1/Personal-Health-Activity-Tracker

Project Overview

Dataset Human Activity Recognition Using Smartphones Dataset https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones

Two Tasks for Machine Learning Based on how the dataset was collected and formatted, the team decided to work on the following tasks where Machine Learning can be applied

1. In real-time, given the most recent 128 sensor readings, classify the human activity
2. In real-time, given the most recent 64 sensor readings, predict the next 64

Part I: Exploratory Data Analysis (EDA) and Pre-processing

Imports

In [1]:
```
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import tensorflow as tf
from importlib import reload
import keras

import numpy as np
from sklearn.preprocessing import LabelEncoder

# KERAS RANDOM SEED FOR REPRODUCIBILITY
keras.utils.set_random_seed(1234)
np.random.seed(1234)
PYTHONHASHSEED = 0
```

Data Loading, Preprocessing, and Analysis

In [2]:
```
# DATASET PATH
DATASET_DIR = '../Dataset'
```

In [3]:
```
# Load Human Activity Labels, and Available Features

# Define file paths
activity_labels_path = f"{DATASET_DIR}/activity_labels.txt"
features_path = f"{DATASET_DIR}/features.txt"

# ✅ Load activity<->label mappings
activity_labels = pd.read_csv(activity_labels_path, header=None, sep=' ', names=['id', 'activity'])
activity_labels.set_index('id', inplace=True)  # Set index for easy mapping

# ✅ Load feature names
features = pd.read_csv(features_path, header=None, sep=' ', names=['id', 'feature_name'])

# Define file paths for train and test data
train_path = f"{DATASET_DIR}/train/"
test_path = f"{DATASET_DIR}/test/"

# ✅ Load subject, X (features), and y (labels) for training
subject_train = pd.read_csv(train_path + "subject_train.txt", header=None, names=['subject'])
X_train = pd.read_csv(train_path + "X_train.txt", sep=r'\s+', header=None)
y_train = pd.read_csv(train_path + "y_train.txt", header=None, names=['activity'])

# ✅ Load subject, X (features), and y (labels) for testing
subject_test = pd.read_csv(test_path + "subject_test.txt", header=None, names=['subject'])
```

```python
X_test = pd.read_csv(test_path + "X_test.txt", sep=r'\s+', header=None)
y_test = pd.read_csv(test_path + "y_test.txt", header=None, names=['activity'])

# ✅ Assign feature names to X_train & X_test
X_train.columns = features["feature_name"]
X_test.columns = features["feature_name"]

# ✅ Convert activity numbers to labels
y_train["activity"] = y_train["activity"].map(activity_labels["activity"])
y_test["activity"] = y_test["activity"].map(activity_labels["activity"])

# ✅ Combine subject, activity, and features into final datasets
train_data = pd.concat([subject_train, y_train, X_train], axis=1)
test_data = pd.concat([subject_test, y_test, X_test], axis=1)


# Display loaded data
print("✅ Human Activity Label Mapping:")
print(activity_labels)

print("\n✅ Features Preview:")
print(features.head())

# Display dataset info
print("🔍 Train Dataset Info:")
train_data.info()
print("\n🔍 Test Dataset Info:")
test_data.info()

# Preview the first few rows
print("\n📌 Train Data Preview:")
display(train_data.head())

print("\n📌 Test Data Preview:")
display(test_data.head())

# Check for missing values
print("\n❓ Missing Values in Train Data:")
print(train_data.isnull().sum().sum())

print("\n❓ Missing Values in Test Data:")
print(test_data.isnull().sum().sum())
```

```
✅ Human Activity Label Mapping:
             activity
id
1              WALKING
2     WALKING_UPSTAIRS
3   WALKING_DOWNSTAIRS
4              SITTING
5             STANDING
6               LAYING

✅ Features Preview:
   id       feature_name
0   1  tBodyAcc-mean()-X
1   2  tBodyAcc-mean()-Y
2   3  tBodyAcc-mean()-Z
3   4   tBodyAcc-std()-X
4   5   tBodyAcc-std()-Y
🔍 Train Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7352 entries, 0 to 7351
Columns: 563 entries, subject to angle(Z,gravityMean)
dtypes: float64(561), int64(1), object(1)
memory usage: 31.6+ MB

🔍 Test Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2947 entries, 0 to 2946
Columns: 563 entries, subject to angle(Z,gravityMean)
dtypes: float64(561), int64(1), object(1)
memory usage: 12.7+ MB

📌 Train Data Preview:
```

	subject	activity	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X	tBodyAcc-mad()-Y	...	fBodyBodyGyroJerkMag-meanFreq()	fBody
0	1	STANDING	0.288585	-0.020294	-0.132905	-0.995279	-0.983111	-0.913526	-0.995112	-0.983185	...	-0.074323	
1	1	STANDING	0.278419	-0.016411	-0.123520	-0.998245	-0.975300	-0.960322	-0.998807	-0.974914	...	0.158075	
2	1	STANDING	0.279653	-0.019467	-0.113462	-0.995380	-0.967187	-0.978944	-0.996520	-0.963668	...	0.414503	
3	1	STANDING	0.279174	-0.026201	-0.123283	-0.996091	-0.983403	-0.990675	-0.997099	-0.982750	...	0.404573	
4	1	STANDING	0.276629	-0.016570	-0.115362	-0.998139	-0.980817	-0.990482	-0.998321	-0.979672	...	0.087753	

5 rows × 563 columns

📌 Test Data Preview:

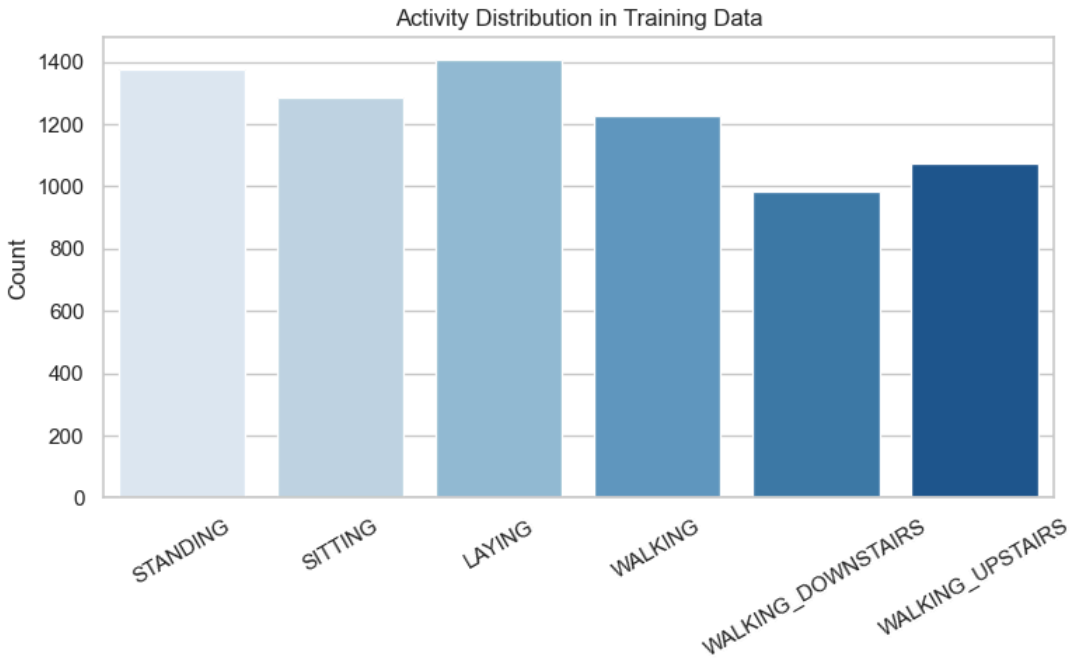
	subject	activity	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X	tBodyAcc-mad()-Y	...	fBodyBodyGyroJerkMag-meanFreq()	fBody
0	2	STANDING	0.257178	-0.023285	-0.014654	-0.938404	-0.920091	-0.667683	-0.952501	-0.925249	...	0.071645	
1	2	STANDING	0.286027	-0.013163	-0.119083	-0.975415	-0.967458	-0.944958	-0.986799	-0.968401	...	-0.401189	
2	2	STANDING	0.275485	-0.026050	-0.118152	-0.993819	-0.969926	-0.962748	-0.994403	-0.970735	...	0.062891	
3	2	STANDING	0.270298	-0.032614	-0.117520	-0.994743	-0.973268	-0.967091	-0.995274	-0.974471	...	0.116695	
4	2	STANDING	0.274833	-0.027848	-0.129527	-0.993852	-0.967445	-0.978295	-0.994111	-0.965953	...	-0.121711	

5 rows × 563 columns

❓ Missing Values in Train Data:
0

❓ Missing Values in Test Data:
0

In [4]:
```
# Suppress warnings
#warnings.simplefilter(action="ignore", category=FutureWarning)
#warnings.simplefilter(action="ignore", category=UserWarning)

# Since the dataset has already been split into TRAIN and TEST sets, perform EDA EXCLUSIVELY on Train Data

# Set style for plots
plt.style.use("ggplot")
sns.set_theme(style="whitegrid")

# Count activity labels in train and test datasets
plt.figure(figsize=(8, 5))

# Training Set
sns.countplot(data=train_data, x="activity", hue="activity", palette="Blues", legend=False)
plt.title("Activity Distribution in Training Data")
plt.xlabel(xlabel=None)
plt.ylabel("Count")
plt.xticks(rotation=30)


plt.tight_layout()
plt.show()
```



In [5]:
```
# Summary statistics of numerical columns
print("📌 Summary Statistics (Train Data):")
display(train_data.describe())

print("\n📌 Summary Statistics (Test Data):")
display(test_data.describe())
```

📌 Summary Statistics (Train Data):

	subject	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X	tBodyAcc-mad()-Y	tBodyAcc-mad()-Z	...	fB
count	7352.000000	7352.000000	7352.000000	7352.000000	7352.000000	7352.000000	7352.000000	7352.000000	7352.000000	7352.000000	...	
mean	17.413085	0.274488	-0.017695	-0.109141	-0.605438	-0.510938	-0.604754	-0.630512	-0.526907	-0.606150	...	
std	8.975143	0.070261	0.040811	0.056635	0.448734	0.502645	0.418687	0.424073	0.485942	0.414122	...	
min	1.000000	-1.000000	-1.000000	-1.000000	-1.000000	-0.999873	-1.000000	-1.000000	-1.000000	-1.000000	...	
25%	8.000000	0.262975	-0.024863	-0.120993	-0.992754	-0.978129	-0.980233	-0.993591	-0.978162	-0.980251	...	
50%	19.000000	0.277193	-0.017219	-0.108676	-0.946196	-0.851897	-0.859365	-0.950709	-0.857328	-0.857143	...	
75%	26.000000	0.288461	-0.010783	-0.097794	-0.242813	-0.034231	-0.262415	-0.292680	-0.066701	-0.265671	...	
max	30.000000	1.000000	1.000000	1.000000	1.000000	0.916238	1.000000	1.000000	0.967664	1.000000	...	

8 rows × 562 columns

📌 Summary Statistics (Test Data):

	subject	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X	tBodyAcc-mad()-Y	tBodyAcc-mad()-Z	...	fBo
count	2947.000000	2947.000000	2947.000000	2947.000000	2947.000000	2947.000000	2947.000000	2947.000000	2947.000000	2947.000000	...	
mean	12.986427	0.273996	-0.017863	-0.108386	-0.613635	-0.508330	-0.633797	-0.641278	-0.522676	-0.637038	...	
std	6.950984	0.060570	0.025745	0.042747	0.412597	0.494269	0.362699	0.385199	0.479899	0.357753	...	
min	2.000000	-0.592004	-0.362884	-0.576184	-0.999606	-1.000000	-0.998955	-0.999417	-0.999914	-0.998899	...	
25%	9.000000	0.262075	-0.024961	-0.121162	-0.990914	-0.973664	-0.976122	-0.992333	-0.974131	-0.975352	...	
50%	12.000000	0.277113	-0.016967	-0.108458	-0.931214	-0.790972	-0.827534	-0.937664	-0.799907	-0.817005	...	
75%	18.000000	0.288097	-0.010143	-0.097123	-0.267395	-0.105919	-0.311432	-0.321719	-0.133488	-0.322771	...	
max	24.000000	0.671887	0.246106	0.494114	0.465299	1.000000	0.489703	0.439657	1.000000	0.427958	...	

8 rows × 562 columns

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
#import warnings


# Display correlation matrix of available features for a specified sensor
```

```
sensor_name='tBodyAcc'

# Suppress font-related warnings
#warnings.simplefilter(action="ignore", category=UserWarning)

# Select a subset of features for correlation analysis
corr_subset = train_data.iloc[:, 2:12].corr()

# Heatmap visualization
plt.figure(figsize=(10, 8))
sns.heatmap(corr_subset, annot=True, cmap="coolwarm", fmt=".2f", square=True)

# Use a simple title to avoid missing glyph warnings
plt.title("Correlation Matrix of Selected Features (Train Data)")

plt.show()
```

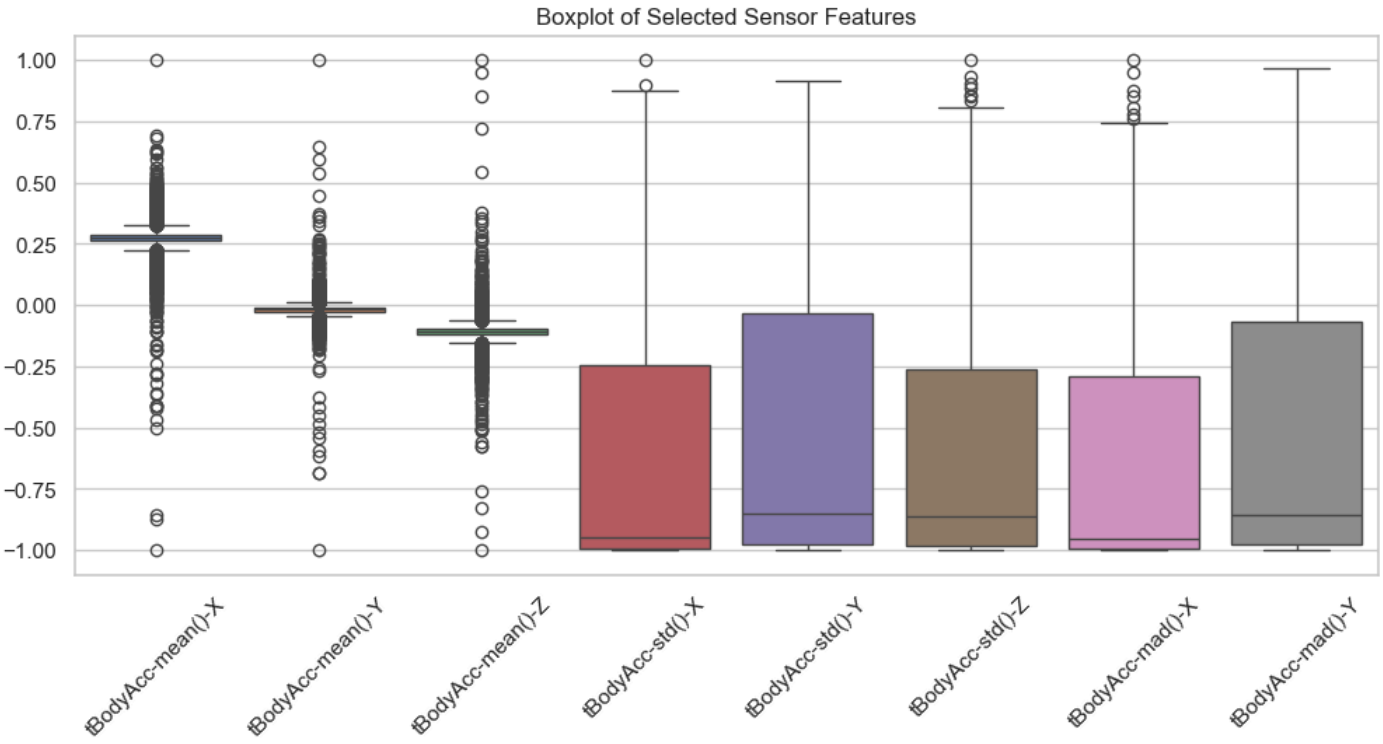
In [7]:
```
train_data.head()
```

Out[7]:

	subject	activity	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X	tBodyAcc-mad()-Y	...	fBodyBodyGyroJerkMag-meanFreq()	fBoc
0	1	STANDING	0.288585	-0.020294	-0.132905	-0.995279	-0.983111	-0.913526	-0.995112	-0.983185	...	-0.074323	
1	1	STANDING	0.278419	-0.016411	-0.123520	-0.998245	-0.975300	-0.960322	-0.998807	-0.974914	...	0.158075	
2	1	STANDING	0.279653	-0.019467	-0.113462	-0.995380	-0.967187	-0.978944	-0.996520	-0.963668	...	0.414503	
3	1	STANDING	0.279174	-0.026201	-0.123283	-0.996091	-0.983403	-0.990675	-0.997099	-0.982750	...	0.404573	
4	1	STANDING	0.276629	-0.016570	-0.115362	-0.998139	-0.980817	-0.990482	-0.998321	-0.979672	...	0.087753	

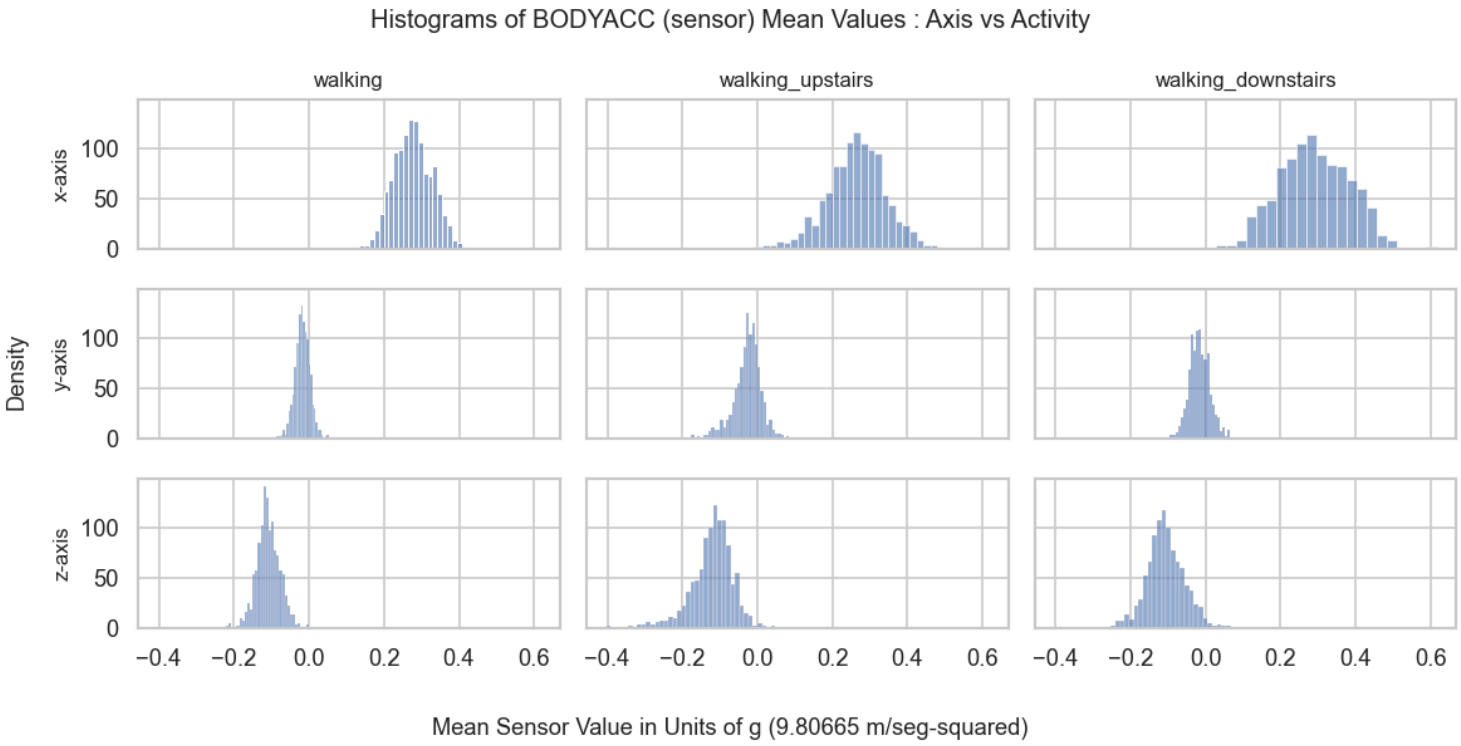
5 rows × 563 columns

In [6]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

# Suppress font-related warnings
warnings.simplefilter(action="ignore", category=UserWarning)

# Select numerical sensor data for boxplot visualization
plt.figure(figsize=(12, 5))
sns.boxplot(data=train_data.iloc[:, 2:10])

# Use a simple title to avoid missing glyph warnings
plt.title("Boxplot of Selected Sensor Features")

plt.xticks(rotation=45)
plt.show()
```



In [15]:
```
# Compare sensor-feature distributions based on activity
# for each of the six activities, compare the per-axis/per-128 sensor reading - MEAN DISTRIBUTION
# CASE 1: MOBILE + BODYACC
# ACTIVITY = MOBILE {'WALKING', 'WALKING_UPSTAIRS', 'WALKING_DOWNSTAIRS'}, SENSOR = BODYACC

actsToAnalyze = ['WALKING', 'WALKING_UPSTAIRS', 'WALKING_DOWNSTAIRS']

sensor_name = 'BodyAcc'
sensor_units = {'BodyAcc' : "Units of g (9.80665 m/seg-squared)",
                'BodyGyro' : "radians/seg"}
sensor_unit = sensor_units[sensor_name]
# feature to analyze: always per-axis mean
sensor_XYZ_features = [f't{sensor_name}-mean()-X',
                       f't{sensor_name}-mean()-Y',
                       f't{sensor_name}-mean()-Z']



print(f'Sensor_XYZ_features: {sensor_XYZ_features}')
fig=[]
axes=[]

fig, axes = plt.subplots(3, len(actsToAnalyze), figsize=(10, 5), dpi=120, sharex=True, sharey=True)

for act_id, activity in enumerate(actsToAnalyze):
    filtered_train_data = train_data[train_data['activity']==activity]
    # set column title to activity label
    axes[0,act_id].set_title(f"{activity.lower()}", fontsize=10)

    for xyz_id, xyz_feature in enumerate(sensor_XYZ_features):
        # display feature stat
        sns.histplot(filtered_train_data[xyz_feature], ax=axes[xyz_id,act_id], fill=True, alpha=0.6)
        axes[xyz_id,0].set_ylabel(f"{xyz_feature[-1].lower()}-axis", fontsize=10)
        axes[xyz_id,act_id].set_xlabel("")

#fig.suptitle(f'Per-Axis MEAN Values for Sensor {sensor_name.upper()}', fontsize=11)
fig.supylabel(f'Density', fontsize=11)
fig.suptitle(f'Histograms of {sensor_name.upper()} (sensor) Mean Values : Axis vs Activity', fontsize=12)
fig.supxlabel(f'Mean Sensor Value in {sensor_unit}', fontsize=11)
plt.tight_layout()

plt.show()
```

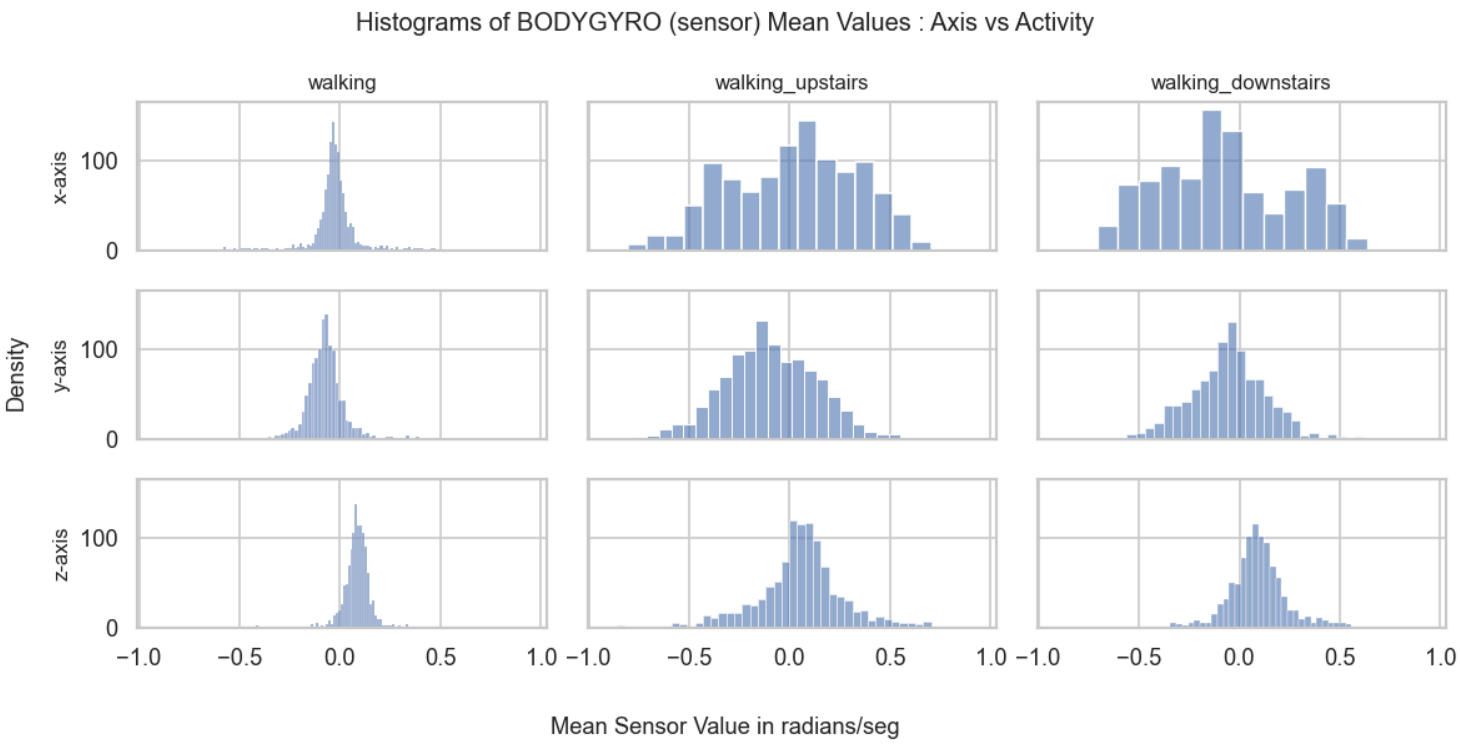
```
Sensor_XYZ_features: ['tBodyAcc-mean()-X', 'tBodyAcc-mean()-Y', 'tBodyAcc-mean()-Z']
```



In [16]:
```
# Compare sensor-feature distributions based on activity
# for each of the six activities, compare the per-axis/per-128 sensor reading - MEAN DISTRIBUTION
# CASE 2: MOBILE + BODYGYRO
# ACTIVITY = MOBILE {'WALKING', 'WALKING_UPSTAIRS', 'WALKING_DOWNSTAIRS'}, SENSOR = BODYACC

actsToAnalyze = ['WALKING', 'WALKING_UPSTAIRS', 'WALKING_DOWNSTAIRS']

sensor_name = 'BodyGyro'
```

```
#sensor_unit = "Units of g (9.80665 m/seg-squared)"
sensor_unit = sensor_units[sensor_name]
# feature to analyze: always per-axis mean
sensor_XYZ_features = [f't{sensor_name}-mean()-X',
                       f't{sensor_name}-mean()-Y',
                       f't{sensor_name}-mean()-Z']



print(f'Sensor_XYZ_features: {sensor_XYZ_features}')
fig=[]
axes=[]

fig, axes = plt.subplots(3, len(actsToAnalyze), figsize=(10, 5), dpi=120, sharex=True, sharey=True)

for act_id, activity in enumerate(actsToAnalyze):
    filtered_train_data = train_data[train_data['activity']==activity]
    # set column title to activity label
    axes[0,act_id].set_title(f"{activity.lower()}", fontsize=10)

    for xyz_id, xyz_feature in enumerate(sensor_XYZ_features):
        # display feature stat
        sns.histplot(filtered_train_data[xyz_feature], ax=axes[xyz_id,act_id], fill=True, alpha=0.6)
        axes[xyz_id,0].set_ylabel(f"{xyz_feature[-1].lower()}-axis", fontsize=10)
        axes[xyz_id,act_id].set_xlabel("")

#fig.suptitle(f'Per-Axis MEAN Values for Sensor {sensor_name.upper()}', fontsize=11)
fig.supylabel(f'Density', fontsize=11)
fig.suptitle(f'Histograms of {sensor_name.upper()} (sensor) Mean Values : Axis vs Activity', fontsize=12)
fig.supxlabel(f'Mean Sensor Value in {sensor_unit}', fontsize=11)
plt.tight_layout()

plt.show()
```

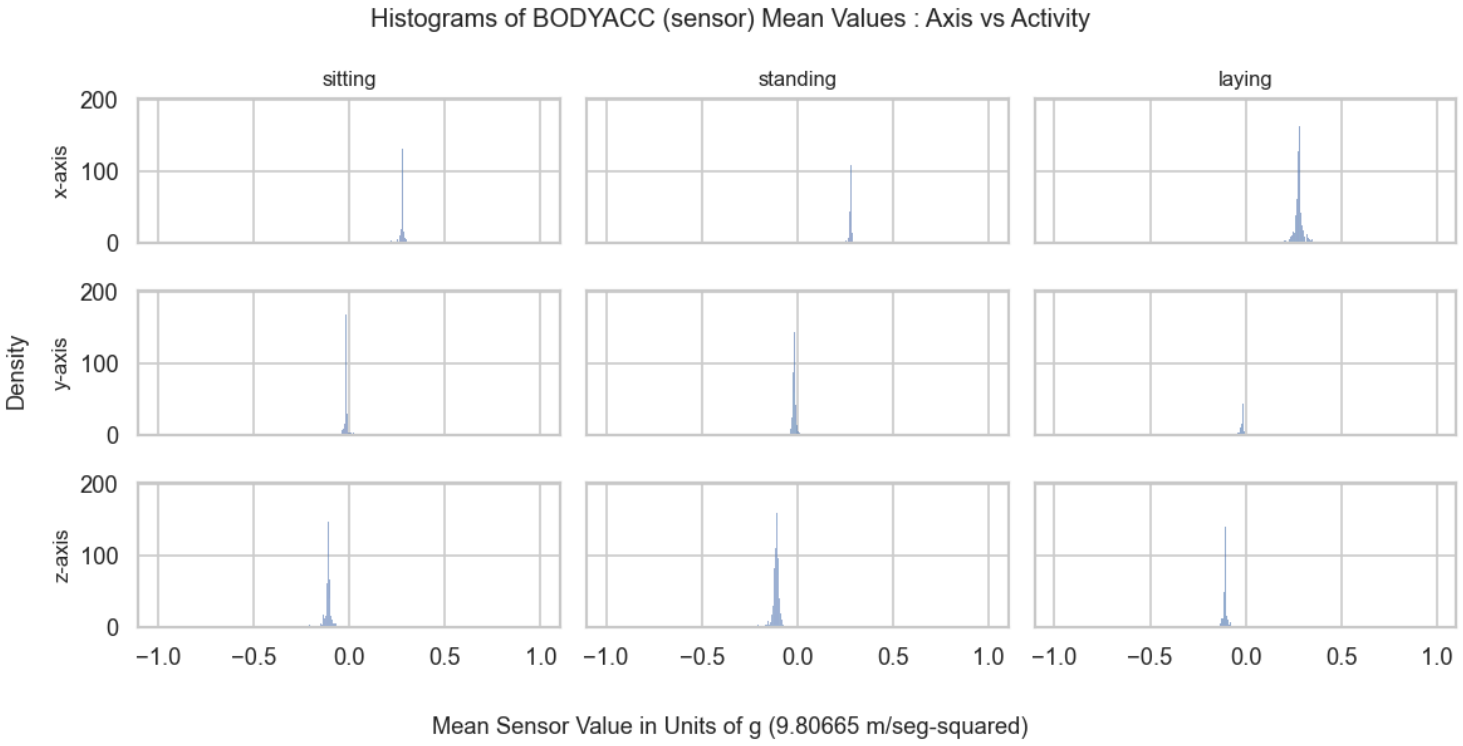
```
Sensor_XYZ_features: ['tBodyGyro-mean()-X', 'tBodyGyro-mean()-Y', 'tBodyGyro-mean()-Z']
```



In [17]:
```
# Compare sensor-feature distributions based on activity
# for each of the six activities, compare the per-axis/per-128 sensor reading - MEAN DISTRIBUTION
# CASE 3: STATIONARY + BODYACC
# ACTIVITY = MOBILE {'WALKING', 'WALKING_UPSTAIRS', 'WALKING_DOWNSTAIRS'}, SENSOR = BODYACC

actsToAnalyze = ['SITTING', 'STANDING', 'LAYING']

sensor_name = 'BodyAcc'
#sensor_unit = "Units of g (9.80665 m/seg-squared)"
sensor_unit = sensor_units[sensor_name]
# feature to analyze: always per-axis mean
sensor_XYZ_features = [f't{sensor_name}-mean()-X',
                       f't{sensor_name}-mean()-Y',
                       f't{sensor_name}-mean()-Z']



print(f'Sensor_XYZ_features: {sensor_XYZ_features}')
```

```
fig=[]
axes=[]

fig, axes = plt.subplots(3, len(actsToAnalyze), figsize=(10, 5), dpi=120, sharex=True, sharey=True)

for act_id, activity in enumerate(actsToAnalyze):
    filtered_train_data = train_data[train_data['activity']==activity]
    # set column title to activity label
    axes[0,act_id].set_title(f"{activity.lower()}", fontsize=10)

    for xyz_id, xyz_feature in enumerate(sensor_XYZ_features):
        # display feature stat
        sns.histplot(filtered_train_data[xyz_feature], ax=axes[xyz_id,act_id], fill=True, alpha=0.6)
        axes[xyz_id,0].set_ylabel(f"{xyz_feature[-1].lower()}-axis", fontsize=10)
        axes[xyz_id,act_id].set_xlabel("")

#fig.suptitle(f'Per-Axis MEAN Values for Sensor {sensor_name.upper()}', fontsize=11)
fig.supylabel(f'Density', fontsize=11)
fig.suptitle(f'Histograms of {sensor_name.upper()} (sensor) Mean Values : Axis vs Activity', fontsize=12)
fig.supxlabel(f'Mean Sensor Value in {sensor_unit}', fontsize=11)
plt.tight_layout()

plt.show()
```

```
Sensor_XYZ_features: ['tBodyAcc-mean()-X', 'tBodyAcc-mean()-Y', 'tBodyAcc-mean()-Z']
```

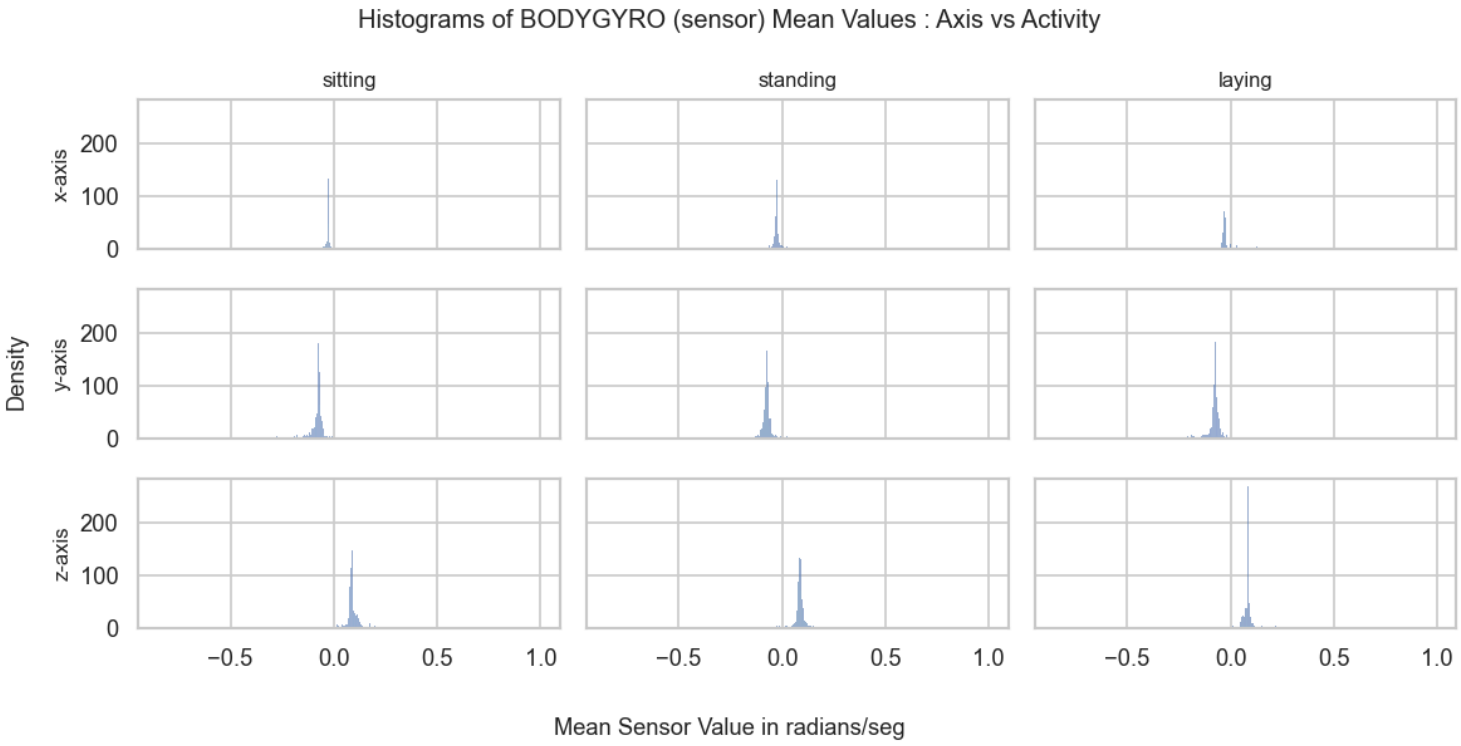


In [18]:

```
# Compare sensor-feature distributions based on activity
# for each of the six activities, compare the per-axis/per-128 sensor reading - MEAN DISTRIBUTION
# CASE 4: STATIONARY + BODYGYRO
# ACTIVITY = MOBILE {'WALKING', 'WALKING_UPSTAIRS', 'WALKING_DOWNSTAIRS'}, SENSOR = BODYACC

actsToAnalyze = ['SITTING', 'STANDING', 'LAYING']

sensor_name = 'BodyGyro'
#sensor_unit = "Units of g (9.80665 m/seg-squared)"
sensor_unit = sensor_units[sensor_name]
# feature to analyze: always per-axis mean
sensor_XYZ_features = [f't{sensor_name}-mean()-X',
                       f't{sensor_name}-mean()-Y',
                       f't{sensor_name}-mean()-Z']



print(f'Sensor_XYZ_features: {sensor_XYZ_features}')
fig=[]
axes=[]

fig, axes = plt.subplots(3, len(actsToAnalyze), figsize=(10, 5), dpi=120, sharex=True, sharey=True)

for act_id, activity in enumerate(actsToAnalyze):
    filtered_train_data = train_data[train_data['activity']==activity]
    # set column title to activity label
    axes[0,act_id].set_title(f"{activity.lower()}", fontsize=10)
```

```
    for xyz_id, xyz_feature in enumerate(sensor_XYZ_features):
        # display feature stat
        sns.histplot(filtered_train_data[xyz_feature], ax=axes[xyz_id,act_id], fill=True, alpha=0.6)
        axes[xyz_id,0].set_ylabel(f"{xyz_feature[-1].lower()}-axis", fontsize=10)
        axes[xyz_id,act_id].set_xlabel("")

#fig.suptitle(f'Per-Axis MEAN Values for Sensor {sensor_name.upper()}', fontsize=11)
fig.supylabel(f'Density', fontsize=11)
fig.suptitle(f'Histograms of {sensor_name.upper()} (sensor) Mean Values : Axis vs Activity', fontsize=12)
fig.supxlabel(f'Mean Sensor Value in {sensor_unit}', fontsize=11)
plt.tight_layout()

plt.show()
```

```
Sensor_XYZ_features: ['tBodyGyro-mean()-X', 'tBodyGyro-mean()-Y', 'tBodyGyro-mean()-Z']
```



In [19]:
```
# remove first two columns of train and test data to create X_train and X_test
# train
X_train = train_data.iloc[:, 2:]
y_train = train_data.iloc[:, 1]
display(X_train.head())
display(y_train.head())

# test
X_test = test_data.iloc[:, 2:]
y_test = test_data.iloc[:, 1]
display(X_test.head())
display(y_test.head())

label_encoder = LabelEncoder()
y_train_encoded = label_encoder.fit_transform(y_train)  # Convert labels to numbers
y_test_encoded = label_encoder.transform(y_test)
```

	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X	tBodyAcc-mad()-Y	tBodyAcc-mad()-Z	tBodyAcc-max()-X	...	fBodyBodyGyroJerkMag-meanFreq()	fB
0	0.288585	-0.020294	-0.132905	-0.995279	-0.983111	-0.913526	-0.995112	-0.983185	-0.923527	-0.934724	...	-0.074323	
1	0.278419	-0.016411	-0.123520	-0.998245	-0.975300	-0.960322	-0.998807	-0.974914	-0.957686	-0.943068	...	0.158075	
2	0.279653	-0.019467	-0.113462	-0.995380	-0.967187	-0.978944	-0.996520	-0.963668	-0.977469	-0.938692	...	0.414503	
3	0.279174	-0.026201	-0.123283	-0.996091	-0.983403	-0.990675	-0.997099	-0.982750	-0.989302	-0.938692	...	0.404573	
4	0.276629	-0.016570	-0.115362	-0.998139	-0.980817	-0.990482	-0.998321	-0.979672	-0.990441	-0.942469	...	0.087753	

5 rows × 561 columns

```
0    STANDING
1    STANDING
2    STANDING
3    STANDING
4    STANDING
Name: activity, dtype: object
```

	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X	tBodyAcc-mad()-Y	tBodyAcc-mad()-Z	tBodyAcc-max()-X	...	fBodyBodyGyroJerkMag-meanFreq()	fB
0	0.257178	-0.023285	-0.014654	-0.938404	-0.920091	-0.667683	-0.952501	-0.925249	-0.674302	-0.894088	...	0.071645	
1	0.286027	-0.013163	-0.119083	-0.975415	-0.967458	-0.944958	-0.986799	-0.968401	-0.945823	-0.894088	...	-0.401189	
2	0.275485	-0.026050	-0.118152	-0.993819	-0.969926	-0.962748	-0.994403	-0.970735	-0.963483	-0.939260	...	0.062891	
3	0.270298	-0.032614	-0.117520	-0.994743	-0.973268	-0.967091	-0.995274	-0.974471	-0.968897	-0.938610	...	0.116695	
4	0.274833	-0.027848	-0.129527	-0.993852	-0.967445	-0.978295	-0.994111	-0.965953	-0.977346	-0.938610	...	-0.121711	

5 rows × 561 columns

```
0    STANDING
1    STANDING
2    STANDING
3    STANDING
4    STANDING
Name: activity, dtype: object
```

Part II: MACHINE LEARNING TASK 1 (HUMAN ACTIVITY CLASSIFICATION)

Part II.a RandomForestClassifier

In [20]:
```
import ClassifierHelperFunctions as chf
reload(chf)
```

Out[20]:
```
<module 'ClassifierHelperFunctions' from '/Users/geoffreyfadera/Documents/USD Stuff/AAI-530/TeamProject/Personal-Health-Activity-Tracker/ClassifierHelperFunctions.py'>
```

In [21]:
```
# Classifier Model 1: Random Forest
model_name = "Random Forest Classifier"
# train model and display learning curves if available
rf_model=[]
rf_model = chf.classifier_RF(X_train, y_train, n_estimators=100)

# predict activity labels on test data
y_pred = chf.classifier_predict(rf_model, X_test, label_encoder=[], model_id=0)

# evaluate predictions against true labels
rf_acc, rf_classreport = chf.classifier_evaluate(y_true=y_test, y_pred=y_pred, model_name=model_name)
```

```
Random Forest Classifier
accuracy score: 0.9237
Classification Report Random Forest Classifier
                    precision    recall  f1-score   support

           LAYING       1.00      1.00      1.00       537
          SITTING       0.90      0.89      0.89       491
         STANDING       0.90      0.91      0.90       532
          WALKING       0.89      0.97      0.93       496
WALKING_DOWNSTAIRS       0.96      0.85      0.90       420
  WALKING_UPSTAIRS       0.91      0.91      0.91       471

          accuracy                           0.92      2947
         macro avg       0.92      0.92      0.92      2947
      weighted avg       0.92      0.92      0.92      2947
```

Part II.b LSTM Classifier

In [22]:
```
# Classifier Model 2: LSTM Classifier
model_name = "LSTM Classifier"

# train model and display learning curves if available
lstm_classifier=[]
lstm_classifier, lstm_classifier_history = chf.classifier_LSTM(X_train, y_train,
                                                               X_test, y_test,
                                                               label_encoder = label_encoder,
                                                               epochs = 10,
                                                               batch_size = 32, #validation_split = 0.2,
                                                               model_name = model_name)

# predict activity labels on test data
# Reshape X_test for LSTM Classifier (samples, time_steps, features)
X_test_lstm = np.array(X_test).reshape((-1, 1, X_test.shape[1]))
y_pred = chf.classifier_predict(lstm_classifier, X_test_lstm, label_encoder=label_encoder, model_id=1)

# evaluate predictions against true labels
lc_acc, lc_classreport = chf.classifier_evaluate(y_true=y_test, y_pred=y_pred, model_name=model_name)
```

Model: "sequential"

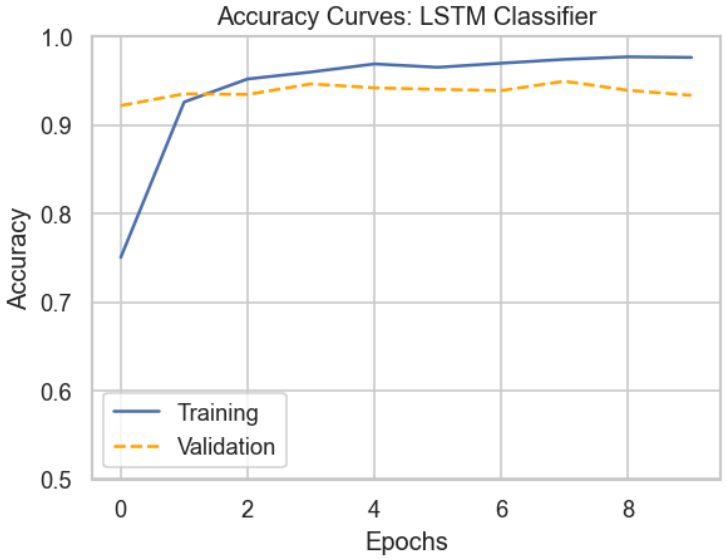
Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 1, 64)	160,256
dropout (Dropout)	(None, 1, 64)	0
lstm_1 (LSTM)	(None, 32)	12,416
dropout_1 (Dropout)	(None, 32)	0
dense (Dense)	(None, 6)	198

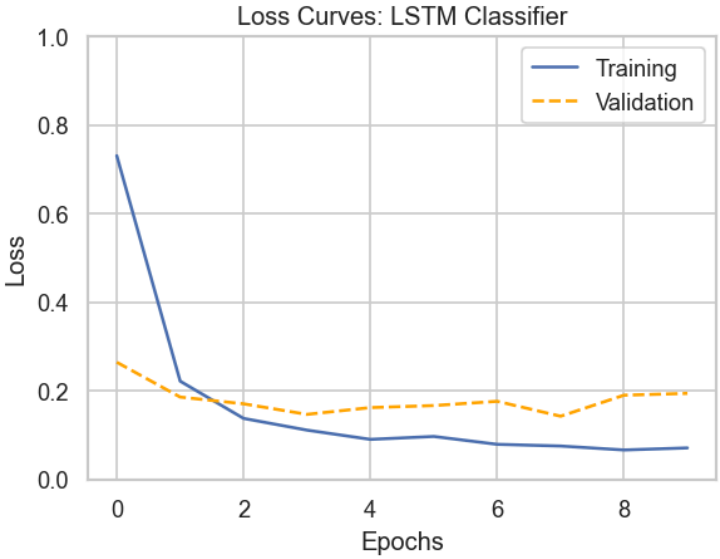
Total params: 172,870 (675.27 KB)

Trainable params: 172,870 (675.27 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5909 - loss: 1.1221 - val_accuracy: 0.9220 - val_loss: 0.2642
Epoch 2/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9191 - loss: 0.2549 - val_accuracy: 0.9352 - val_loss: 0.1854
Epoch 3/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9465 - loss: 0.1509 - val_accuracy: 0.9345 - val_loss: 0.1702
Epoch 4/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9592 - loss: 0.1122 - val_accuracy: 0.9464 - val_loss: 0.1463
Epoch 5/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9699 - loss: 0.0921 - val_accuracy: 0.9420 - val_loss: 0.1617
Epoch 6/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9662 - loss: 0.0928 - val_accuracy: 0.9403 - val_loss: 0.1662
Epoch 7/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9724 - loss: 0.0752 - val_accuracy: 0.9389 - val_loss: 0.1759
Epoch 8/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9756 - loss: 0.0721 - val_accuracy: 0.9494 - val_loss: 0.1422
Epoch 9/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9803 - loss: 0.0619 - val_accuracy: 0.9393 - val_loss: 0.1897
Epoch 10/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9792 - loss: 0.0646 - val_accuracy: 0.9335 - val_loss: 0.1937
```






```
93/93 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
LSTM Classifier
accuracy score: 0.9335
Classification Report LSTM Classifier
                    precision    recall  f1-score   support

           LAYING       1.00      1.00      1.00       537
          SITTING       0.88      0.94      0.91       491
         STANDING       0.95      0.88      0.91       532
          WALKING       0.92      0.98      0.95       496
WALKING_DOWNSTAIRS       0.99      0.87      0.93       420
  WALKING_UPSTAIRS       0.89      0.92      0.90       471

         accuracy                           0.93      2947
        macro avg       0.94      0.93      0.93      2947
     weighted avg       0.94      0.93      0.93      2947
```

Part II.c CNN Classifier

In [23]:
```
# Classifier Model 3: CNN Classifier
model_name = "CNN Classifier"
```

```
# train model and display learning curves if available
cnn_classifier=[]
cnn_classifier, cnn_classifier_history = chf.classifier_CNN(X_train, y_train,
                                                            X_test, y_test,
                                                            label_encoder = label_encoder,
                                                            epochs = 10,
                                                            batch_size = 32, #validation_split = 0.2,
                                                            model_name = model_name)

# predict activity labels on test data
# Reshape Data for CNN Classifier (samples, features)
y_pred = chf.classifier_predict(cnn_classifier, X_test, label_encoder=label_encoder, model_id=2)

# evaluate predictions against true labels
cnn_acc, cnn_classreport = chf.classifier_evaluate(y_true=y_test, y_pred=y_pred, model_name=model_name)
```

Model: "sequential_1"

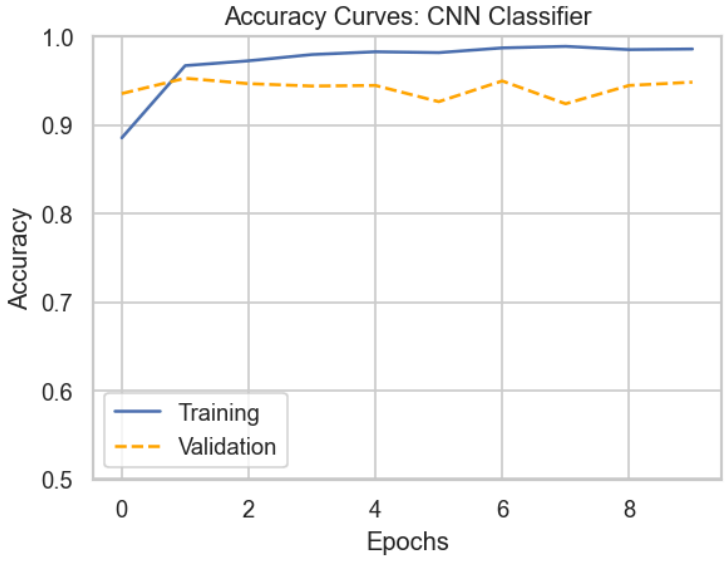
Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 559, 64)	256
conv1d_1 (Conv1D)	(None, 557, 64)	12,352
dropout_2 (Dropout)	(None, 557, 64)	0
max_pooling1d (MaxPooling1D)	(None, 278, 64)	0
flatten (Flatten)	(None, 17792)	0
dense_1 (Dense)	(None, 100)	1,779,300
dense_2 (Dense)	(None, 6)	606

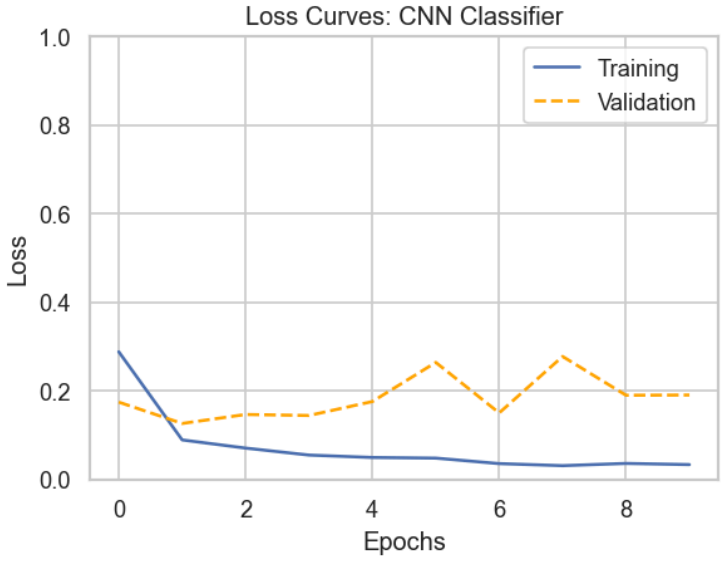
Total params: 1,792,514 (6.84 MB)

Trainable params: 1,792,514 (6.84 MB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 4s 17ms/step - accuracy: 0.7673 - loss: 0.5768 - val_accuracy: 0.9355 - val_loss: 0.1740
Epoch 2/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - accuracy: 0.9659 - loss: 0.0966 - val_accuracy: 0.9528 - val_loss: 0.1258
Epoch 3/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 4s 15ms/step - accuracy: 0.9725 - loss: 0.0706 - val_accuracy: 0.9467 - val_loss: 0.1461
Epoch 4/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - accuracy: 0.9825 - loss: 0.0496 - val_accuracy: 0.9440 - val_loss: 0.1438
Epoch 5/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - accuracy: 0.9821 - loss: 0.0534 - val_accuracy: 0.9447 - val_loss: 0.1753
Epoch 6/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 4s 15ms/step - accuracy: 0.9829 - loss: 0.0432 - val_accuracy: 0.9264 - val_loss: 0.2637
Epoch 7/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - accuracy: 0.9896 - loss: 0.0302 - val_accuracy: 0.9498 - val_loss: 0.1496
Epoch 8/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - accuracy: 0.9898 - loss: 0.0274 - val_accuracy: 0.9240 - val_loss: 0.2772
Epoch 9/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 4s 15ms/step - accuracy: 0.9852 - loss: 0.0349 - val_accuracy: 0.9447 - val_loss: 0.1895
Epoch 10/10
230/230 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - accuracy: 0.9881 - loss: 0.0269 - val_accuracy: 0.9484 - val_loss: 0.1901
```





```
93/93 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
CNN Classifier
accuracy score: 0.9484
Classification Report CNN Classifier
                    precision    recall  f1-score   support

           LAYING       1.00      1.00      1.00       537
          SITTING       1.00      0.82      0.90       491
         STANDING       0.86      0.99      0.92       532
          WALKING       0.95      0.98      0.97       496
WALKING_DOWNSTAIRS       0.99      0.93      0.96       420
  WALKING_UPSTAIRS       0.92      0.95      0.94       471

          accuracy                           0.95      2947
         macro avg       0.95      0.95      0.95      2947
      weighted avg       0.95      0.95      0.95      2947
```

Classifier Performance Analysis

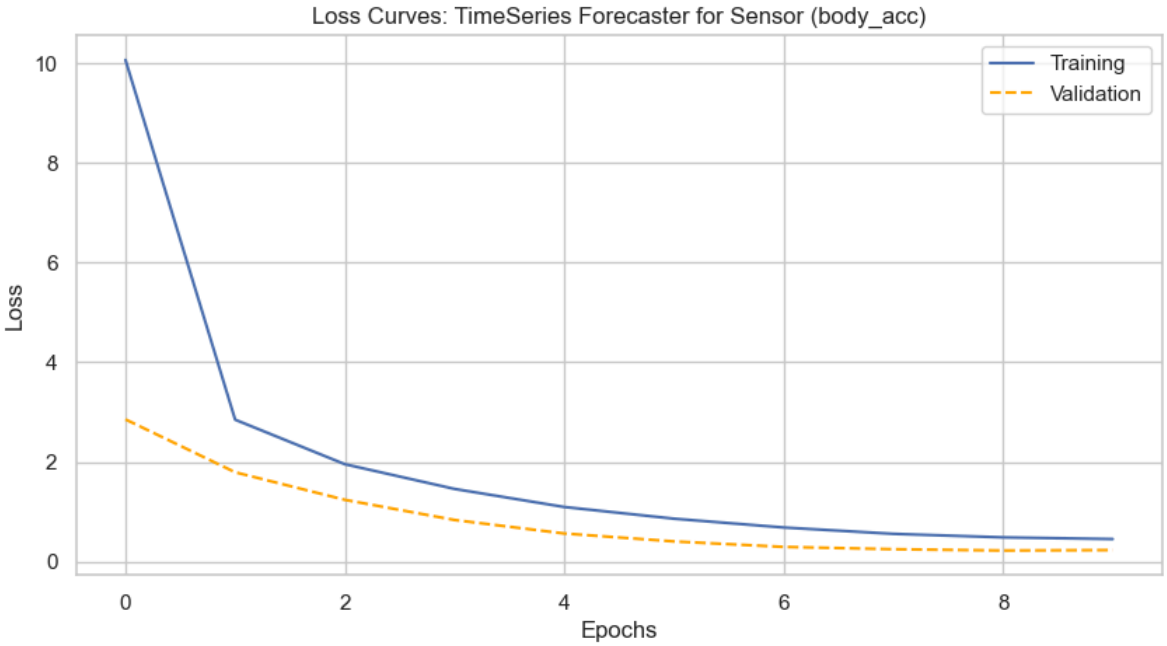
The deep learning classifiers (LSTM & CNN) outperformed the Random Forest Classifier. The learning curves for both LSTM and CNN classifiers show overfitting but not by any significant margin. Based on the f1-scores per activity and total accuracy, CNN is the best performing model. We have decided to use this classifier model on our Dashboard

Part III: MACHINE LEARNING TASK 2 (TIME-SERIES PREDICTION)

In [24]:
```
import TimeSeriesHelperFunctions as tshf
reload(tshf)

TS_NUM_EPOCHS = 10
TS_BATCH_SIZE = 500
TS_VAL_SPLIT = 0.2
```

In [25]:
```
# time-series predictor for sensor BODY_ACC
sensor_name = 'body_acc' #, 'body_gyro', 'total_acc']
loss_function = 'trace_mse_loss'
model_name = f"Time-Series Forecaster Using the Sensor {sensor_name}"
ts_metrics_on_test1 = {}

# predictor
ts_model1, ts_history1, ts_sensor_scalers1, ts_metrics_on_test1 = tshf.time_series_predictor_pipeline(
    root_dir = DATASET_DIR,
    sensor_name = sensor_name,
    model_name = model_name,
    user_loss_fn = loss_function,
    num_epochs= TS_NUM_EPOCHS,
    batch_size=TS_BATCH_SIZE,
    val_split=TS_VAL_SPLIT,
    debug_mode = True)

print("/n")
print(f"TimeSeries Forecaster Evaluation Metrics for Sensor {sensor_name}")

pd.DataFrame(ts_metrics_on_test1)
```

```
TimeSeriesPredictor for SENSOR: body_acc and MODEL: Time-Series Forecaster Using the Sensor body_acc
TS_STEP 1: Load train xyz data for SENSOR (body_acc)
TS_STEP 2: scale train xyz data per x,y,z axis using MinMaxScaler
TS_STEP 3: split scaled train xyz data into two along the time-axis (axis=1)
TS_STEP 4: Create, train, and save a TimeSeriesPredictor
Epoch 1/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 9s 451ms/step - loss: 14.1067 - mae: 0.4343 - mse: 0.2115 - val_loss: 2.8537 - val_mae: 0.0843 - val_mse:
0.0162
Epoch 2/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 402ms/step - loss: 3.0430 - mae: 0.1161 - mse: 0.0235 - val_loss: 1.7913 - val_mae: 0.0759 - val_mse:
0.0115
Epoch 3/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 6s 481ms/step - loss: 2.0630 - mae: 0.1007 - mse: 0.0176 - val_loss: 1.2378 - val_mae: 0.0555 - val_mse:
0.0078
Epoch 4/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 419ms/step - loss: 1.5444 - mae: 0.0912 - mse: 0.0143 - val_loss: 0.8326 - val_mae: 0.0435 - val_mse:
0.0055
Epoch 5/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 440ms/step - loss: 1.1441 - mae: 0.0826 - mse: 0.0118 - val_loss: 0.5615 - val_mae: 0.0535 - val_mse:
0.0053
Epoch 6/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 416ms/step - loss: 0.8947 - mae: 0.0792 - mse: 0.0107 - val_loss: 0.4019 - val_mae: 0.0409 - val_mse:
0.0040
Epoch 7/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 424ms/step - loss: 0.7078 - mae: 0.0729 - mse: 0.0091 - val_loss: 0.2921 - val_mae: 0.0411 - val_mse:
0.0037
Epoch 8/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 6s 542ms/step - loss: 0.5675 - mae: 0.0685 - mse: 0.0081 - val_loss: 0.2456 - val_mae: 0.0396 - val_mse:
0.0035
Epoch 9/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 7s 560ms/step - loss: 0.4879 - mae: 0.0643 - mse: 0.0073 - val_loss: 0.2193 - val_mae: 0.0361 - val_mse:
0.0033
Epoch 10/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 6s 506ms/step - loss: 0.4487 - mae: 0.0615 - mse: 0.0068 - val_loss: 0.2281 - val_mae: 0.0410 - val_mse:
0.0036
dict_keys(['loss', 'mae', 'mse', 'val_loss', 'val_mae', 'val_mse'])
TS_STEP 5: Display loss curves
```



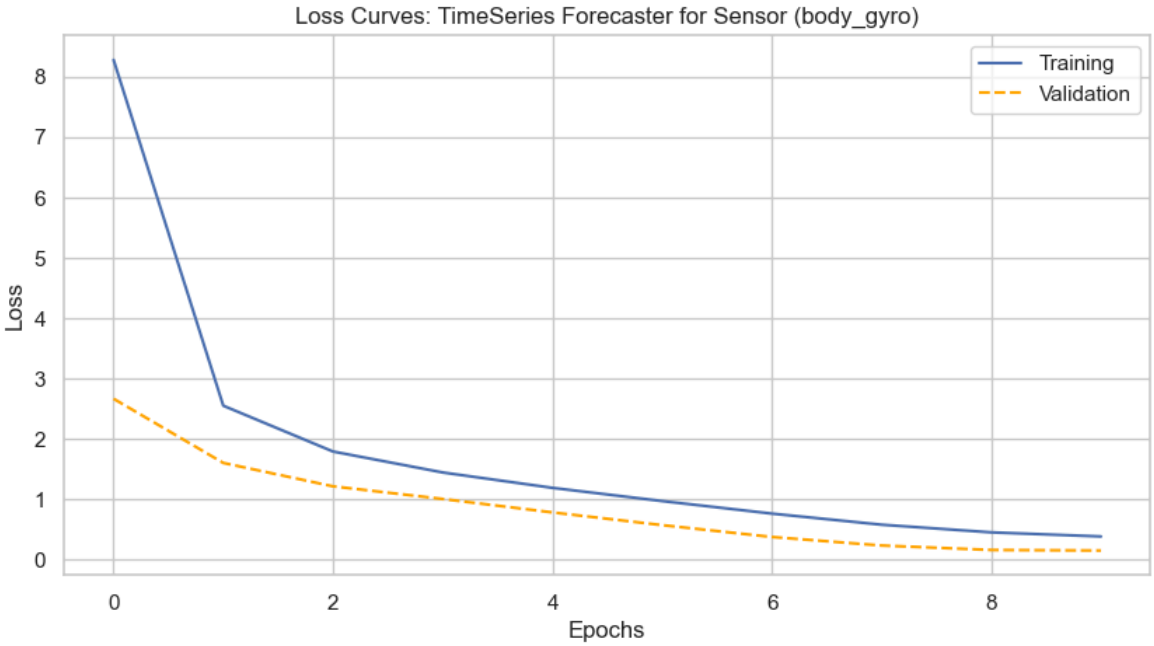
```
TS_STEP 6: Load test xyz data for SENSOR body_acc
TS_STEP 7: Evaluate model on test data
93/93 - 2s - 22ms/step - loss: 0.2321 - mae: 0.0419 - mse: 0.0037
Metrics on Scaled Test Data:
    MSE (scaled): 0.2321
    RMSE (scaled): 0.4818
    MAE (scaled): 0.0037
93/93 ━━━━━━━━━━━━━━━━━━━━ 3s 28ms/step
PRED SHAPE: (2947, 64, 3)
Metrics on Unscaled Test Data:
  x-axis:
    MSE: 0.0355
    RMSE: 0.1884
    MAE: 0.1187
    'TraceMSE': 2.2516
  y-axis:
    MSE: 0.0184
    RMSE: 0.1358
    MAE: 0.1036
    'TraceMSE': 1.1846
  z-axis:
    MSE: 0.0120
    RMSE: 0.1097
    MAE: 0.0832
    'TraceMSE': 0.7410
  xyz-Combined:
    MSE: 0.0220
    RMSE: 0.1483
    MAE: 0.1019
    'TraceMSE': 1.3924
/n
TimeSeries Forecaster Evaluation Metrics for Sensor body_acc
```

Out[25]:

	x-axis	y-axis	z-axis	xyz-Combined
MSE	0.035505	0.018429	0.012027	0.021987
RMSE	0.188427	0.135755	0.109668	0.148281
MAE	0.118676	0.103637	0.083247	0.101853
TraceMSE	2.251556	1.184557	0.740966	1.392360

In [26]:
```python
# time-series predictor for sensor BODY_GYRO
sensor_name = 'body_gyro' #, 'body_gyro', 'total_acc']
loss_function = 'trace_mse_loss'
model_name = f"Time-Series Forecaster Using the Sensor {sensor_name}"
ts_metrics_on_test2 = {}

# predictor
ts_model2, ts_history2, ts_sensor_scalers2, ts_metrics_on_test2 = tshf.time_series_predictor_pipeline(
    root_dir = DATASET_DIR,
    sensor_name = sensor_name,
    model_name = model_name,
    user_loss_fn = loss_function,
    num_epochs= TS_NUM_EPOCHS,
    batch_size=TS_BATCH_SIZE,
    val_split=TS_VAL_SPLIT,
    debug_mode = False)

print("/n")
print(f"TimeSeries Forecaster Evaluation Metrics for Sensor {sensor_name}")

pd.DataFrame(ts_metrics_on_test2)
```

```
TimeSeriesPredictor for SENSOR: body_gyro and MODEL: Time-Series Forecaster Using the Sensor body_gyro
TS_STEP 1: Load train xyz data for SENSOR (body_gyro)
TS_STEP 2: scale train xyz data per x,y,z axis using MinMaxScaler
TS_STEP 3: split scaled train xyz data into two along the time-axis (axis=1)
TS_STEP 4: Create, train, and save a TimeSeriesPredictor
Epoch 1/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 10s 476ms/step - loss: 11.9516 - mae: 0.3947 - mse: 0.1772 - val_loss: 2.6623 - val_mae: 0.0795 - val_mse:
0.0143
Epoch 2/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 426ms/step - loss: 2.7801 - mae: 0.1083 - mse: 0.0204 - val_loss: 1.5982 - val_mae: 0.0931 - val_mse:
0.0126
Epoch 3/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 429ms/step - loss: 1.8678 - mae: 0.0984 - mse: 0.0164 - val_loss: 1.2102 - val_mae: 0.0544 - val_mse:
0.0068
Epoch 4/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 6s 443ms/step - loss: 1.4872 - mae: 0.0890 - mse: 0.0133 - val_loss: 1.0006 - val_mae: 0.0394 - val_mse:
0.0052
Epoch 5/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 446ms/step - loss: 1.2284 - mae: 0.0781 - mse: 0.0106 - val_loss: 0.7781 - val_mae: 0.0362 - val_mse:
0.0043
Epoch 6/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 414ms/step - loss: 1.0056 - mae: 0.0732 - mse: 0.0092 - val_loss: 0.5671 - val_mae: 0.0329 - val_mse:
0.0035
Epoch 7/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 452ms/step - loss: 0.7955 - mae: 0.0701 - mse: 0.0082 - val_loss: 0.3703 - val_mae: 0.0353 - val_mse:
0.0029
Epoch 8/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 412ms/step - loss: 0.6026 - mae: 0.0670 - mse: 0.0074 - val_loss: 0.2291 - val_mae: 0.0396 - val_mse:
0.0027
Epoch 9/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 5s 428ms/step - loss: 0.4604 - mae: 0.0630 - mse: 0.0065 - val_loss: 0.1552 - val_mae: 0.0336 - val_mse:
0.0022
Epoch 10/10
12/12 ━━━━━━━━━━━━━━━━━━━━ 6s 459ms/step - loss: 0.3842 - mae: 0.0589 - mse: 0.0057 - val_loss: 0.1461 - val_mae: 0.0338 - val_mse:
0.0022
dict_keys(['loss', 'mae', 'mse', 'val_loss', 'val_mae', 'val_mse'])
TS_STEP 5: Display loss curves
```




```
TS_STEP 6: Load test xyz data for SENSOR body_gyro
TS_STEP 7: Evaluate model on test data
93/93 - 2s - 22ms/step - loss: 0.1422 - mae: 0.0334 - mse: 0.0022
93/93 ━━━━━━━━━━━━━━━━━━━━ 3s 30ms/step
PRED SHAPE: (2947, 64, 3)
/n
TimeSeries Forecaster Evaluation Metrics for Sensor body_gyro
```

Out[26]:

	x-axis	y-axis	z-axis	xyz-Combined
MSE	0.170138	0.187729	0.078081	0.145316
RMSE	0.412478	0.433277	0.279430	0.381204
MAE	0.286852	0.327224	0.205940	0.273339
TraceMSE	10.609845	13.079007	5.186324	9.625059

Part IV: Dashboard Data Preparation

The project's primary objective is to display activity statistics/information on ANY new user based on the readings from the accelerometer and gyroscope sensors available. It only makes sense to present a Dashboard of human activity metrics based on data from ONE specific person only. The following code section demonstrates this concept.

In [27]:
```
import TimeSeriesHelperFunctions as tshf
reload(tshf)

import ClassifierHelperFunctions as chf
reload(chf)
```

Out[27]:
```
<module 'ClassifierHelperFunctions' from '/Users/geoffreyfadera/Documents/USD Stuff/AAI-530/TeamProject/Personal-Health-Activity-Tracker/ClassifierHelperFunctions.py'>
```

In [28]:
```
# Specify subject id from Test Set to be used on the dashboard
dashboard_human_id = 20
```

In [29]:
```
# Classifier
# filter test data and use only those rows matching 'dashboard_human_id'
test_data_person = test_data[test_data['subject']== dashboard_human_id]
X_test_person = test_data_person.iloc[:, 2:]
y_test_person = test_data_person.iloc[:, 1]

# predict the activity label given a human subject
y_pred_person = chf.classifier_predict(cnn_classifier, X_test_person, label_encoder=label_encoder, model_id=2)
display(y_pred_person.shape)

# save to csv
y_pred_person_df = pd.DataFrame(y_pred_person, columns=["Activity"])
y_pred_person_df.to_csv("predicted_activities_v3.csv", index=True)
```

```
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
(354,)
```

In [30]:
```
# get lstm predictions on sensor body_acc
sensor_name = 'body_acc'

test_sensor_XYZ, test_activity, test_subject = tshf.load_sensor_label_data(root_dir=DATASET_DIR, train_test = 'test', sensor_name=ser
test_sensor_XYZ_person = test_sensor_XYZ[np.where(test_subject==dashboard_human_id)[0], :, :]
display(test_sensor_XYZ_person.shape)

ts_metrics_person, ts_y_pred_person, ts_y_true_person, ts_X_test_person = tshf.evaluate_XYZ_predictor(
        ts_model1, # trained model
        test_sensor_XYZ_person,
        ts_sensor_scalers1, #MinMaxScaler fitted on Train data
        debug_mode=False
        )

pd.DataFrame(ts_metrics_person)
```

```
(354, 128, 3)
12/12 - 0s - 21ms/step - loss: 0.2832 - mae: 0.0455 - mse: 0.0046
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step
PRED SHAPE: (354, 64, 3)
```

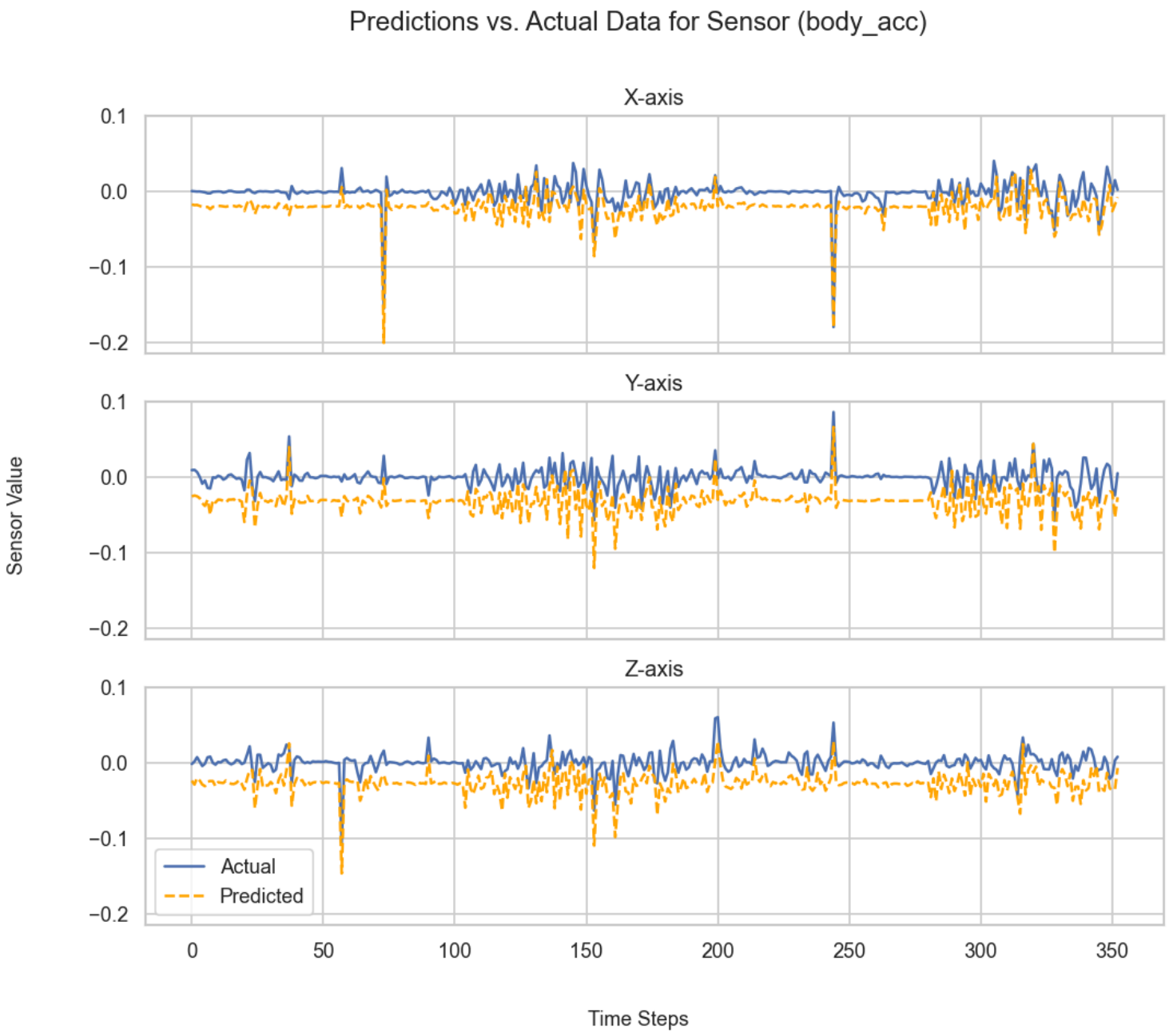
Out[30]:

	x-axis	y-axis	z-axis	xyz-Combined
MSE	0.039141	0.029526	0.012342	0.027003
RMSE	0.197841	0.171831	0.111097	0.164326
MAE	0.123103	0.123754	0.083902	0.110253
TraceMSE	2.420517	1.879752	0.728683	1.676317

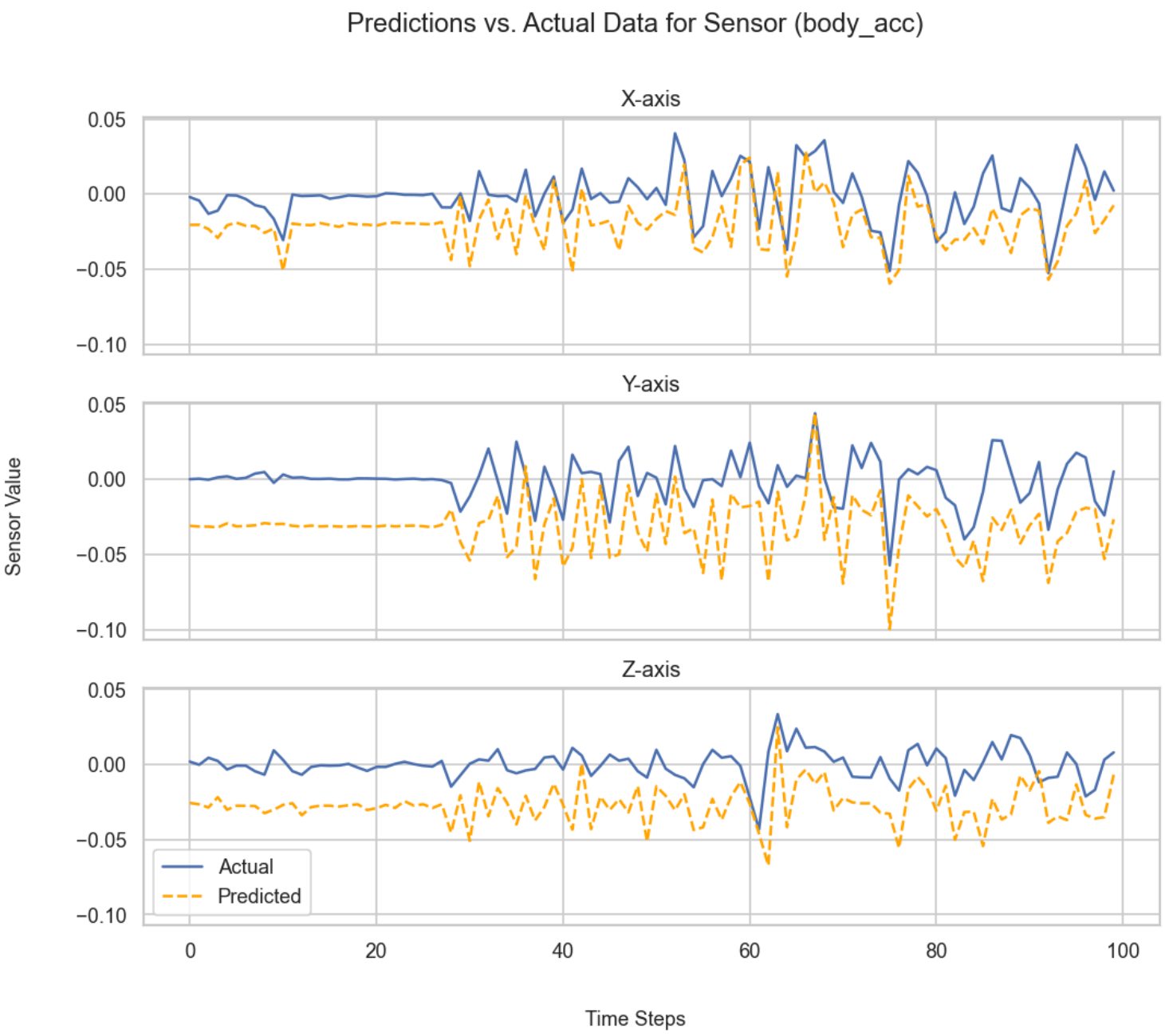
In [31]:
```
# function to compare predicted and actual time-series values
def display_time_series_pred_vs_actual(y_true, y_pred, sensor_name, prev_samples=-1, use_mean=True):


    fig, axes = plt.subplots(3, 1, figsize=(10, 8), dpi=120, sharex=True, sharey=True)
    for axis in range(3):
        if(use_mean==True):
            y_true_axis = np.mean(y_true[:,:,axis], axis=1)
            y_pred_axis = np.mean(y_pred[:,:,axis], axis=1)
        else:
            y_true_axis = y_true[:,:,axis].flatten()
            y_pred_axis = y_pred[:,:,axis].flatten()

        if prev_samples == '-1':
            prev_samples = len(y_true_axis)

        axes[axis].plot(y_true_axis[-prev_samples:], label=f"Actual")
        axes[axis].plot(y_pred_axis[-prev_samples:], label=f"Predicted", linestyle="dashed", color="orange")

        axes[axis].set_title(f"{chr(88+axis)}-axis")
        #axes[axis].set_xlabel("Time Steps")
```

```
        #axes[axis].set_ylabel("Sensor Value")

    fig.supxlabel(f'Time Steps', fontsize=11)
    fig.supylabel(f'Sensor Value', fontsize=11)
    fig.suptitle(f"Predictions vs. Actual Data for Sensor ({sensor_name})")
    plt.legend()
    plt.show()
```

In [32]:
```
# create per 128 TRUE sensor readings, and PREDICTED sensor readings
true_acc_128 = np.concatenate((ts_X_test_person, ts_y_true_person), axis = 1)
pred_acc_128 = np.concatenate((ts_X_test_person, ts_y_pred_person), axis = 1)
display_time_series_pred_vs_actual(true_acc_128, pred_acc_128, sensor_name, use_mean=True)
```



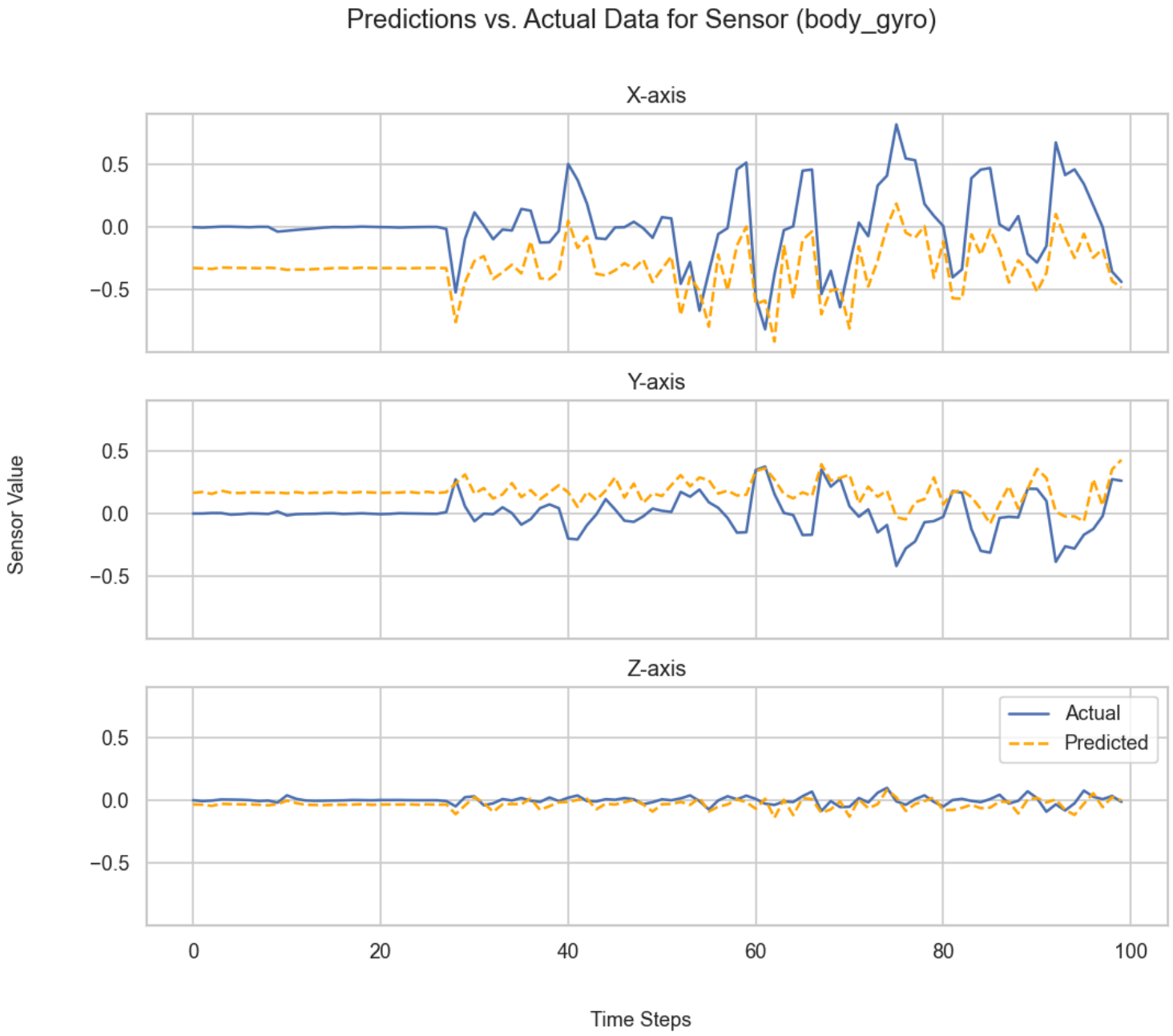
In [33]:
```
display_time_series_pred_vs_actual(true_acc_128, pred_acc_128, sensor_name, prev_samples=100, use_mean=True)
```



In [34]:
```
sensor_name = 'body_gyro'

test_sensor_XYZ, test_activity, test_subject = tshf.load_sensor_label_data(root_dir=DATASET_DIR, train_test = 'test', sensor_name=ser
test_sensor_XYZ_person = test_sensor_XYZ[np.where(test_subject==dashboard_human_id)[0], :, :]
display(test_sensor_XYZ_person.shape)

ts_metrics_person2, ts_y_pred_person2, ts_y_true_person2, ts_X_test_person2 = tshf.evaluate_XYZ_predictor(
        ts_model1, # trained model
        test_sensor_XYZ_person,
        ts_sensor_scalers2, #MinMaxScaler fitted on Train data
        debug_mode=False
        )

display(pd.DataFrame(ts_metrics_person2))

# create per 128 TRUE sensor readings, and PREDICTED sensor readings
true_gyro_128 = np.concatenate((ts_X_test_person2, ts_y_true_person2), axis = 1)
pred_gyro_128 = np.concatenate((ts_X_test_person2, ts_y_pred_person2), axis = 1)
display_time_series_pred_vs_actual(true_gyro_128, pred_gyro_128, sensor_name, prev_samples=100, use_mean=True)
```

```
(354, 128, 3)
12/12 - 0s - 19ms/step - loss: 0.3267 - mae: 0.0539 - mse: 0.0051
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step
PRED SHAPE: (354, 64, 3)
```

	x-axis	y-axis	z-axis	xyz-Combined
MSE	0.624521	0.364217	0.121863	0.370200
RMSE	0.790266	0.603504	0.349089	0.608441
MAE	0.705902	0.474766	0.214552	0.465073
TraceMSE	41.033166	22.814484	7.750465	23.866038



In [35]:
```
# create per 128 TRUE sensor readings, and PREDICTED sensor readings
#np.mean(y_true[:,:,axis], axis=1)

true_acc_128_mean = np.mean(true_acc_128,axis=1)
pred_acc_128_mean = np.mean(pred_acc_128,axis=1)
true_gyro_128_mean = np.mean(true_gyro_128,axis=1)
pred_gyro_128_mean = np.mean(pred_gyro_128,axis=1)
display(true_acc_128_mean.shape)

# save to csv
sensor_means_with_preds_df = pd.DataFrame({
                                        'Acc True-X': true_acc_128_mean[:,0],
                                        'Acc True-Y': true_acc_128_mean[:,1],
                                        'Acc True-Z': true_acc_128_mean[:,2],
                                        'Acc Pred-X': pred_acc_128_mean[:,0],
                                        'Acc Pred-Y': pred_acc_128_mean[:,1],
                                        'Acc Pred-Z': pred_acc_128_mean[:,2],
                                        'Gyro True-X': true_gyro_128_mean[:,0],
                                        'Gyro True-Y': true_gyro_128_mean[:,1],
                                        'Gyro True-Z': true_gyro_128_mean[:,2],
```

```
                                        'Gyro Pred-X': pred_gyro_128_mean[:,0],
                                        'Gyro Pred-Y': pred_gyro_128_mean[:,1],
                                        'Gyro Pred-Z': pred_gyro_128_mean[:,2]})

sensor_means_with_preds_df.to_csv("sensor_means_with_preds_v3.csv", index=True)
```

```
(354, 3)
```

Personal-Health-Activity-Tracker/ClassifierHelperFunctions.py

```python
# Spring 25 AAI-530 Group 5
# Helper Functions for Human Activity Classifier Task

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from keras.models import Sequential
from keras.layers import Dense, Dropout, LSTM
from keras.layers import Conv1D, MaxPooling1D, Flatten
import matplotlib.pyplot as plt
import numpy as np
from keras.utils import to_categorical



def classifier_predict(model, X_test, label_encoder=[], model_id=0):
# predicts the human activity given a 561-feature vector from the most recent 128 sensor readings
# INPUT:
#   model: trained on input of shape (:, X_test[1])
#   X_test: sequence of vectors, each of 561 features derived from the most recent 128 sensor readings

# OUTPUT
#   y_test: predicted human activity label, encoded to strings based on the input label_encoder

    # Prediction
    if(model_id==0): # RF Classifier, output is already categorical
        y_pred = model.predict(X_test)
        model_name = "Random Forest Classifier"
    elif(model_id==1): # LSTM Classifier, output is sparse categorical integer
        y_pred_probs = model.predict(X_test)
        y_pred = label_encoder.inverse_transform(np.argmax(y_pred_probs, axis=1))
        model_name = "LSTM CLassifier"
    elif(model_id==2): # CNN Classifier, output is one-hot encoded 6-bit label
        y_pred_probs = model.predict(X_test)
        y_pred = label_encoder.inverse_transform(np.argmax(y_pred_probs, axis=1))
        model_name = "CNN CLassifier"
    else:
        y_pred = model.predict(X_test)


    if(len(X_test)==1):
        print(f"SINGLE SAMPLE PREDICTION USING : {model_name}")


    return y_pred


def classifier_evaluate(y_true, y_pred, model_name="specify classifier name"):
# evaluates a trained classifier model against the input test data
# INPUT:
#   model: trained on input of shape (:, X_test[1])
#   X_test: test data to evaluate the 'model' on. each sample must be of same shape as each sample in X_train
#   y_test: array of categorical human activity labels of the test data
#
# OUTPUT
#   accuracy_score:
#   classification report

    print(f"{model_name}")

    # Accuracy Score
    accuracy = accuracy_score(y_true, y_pred)
    print(f"accuracy score: {accuracy:.4f}")

    # Display classification report
    class_report = classification_report(y_true, y_pred)
    print(f"Classification Report {model_name}")
    print(class_report)

    return accuracy, class_report




def classifier_RF(X_train, y_train, n_estimators=100):
# Classifier Model: Random Forest
```

```
# INPUT:
#   X_train: array of floats of shape (number of samples, number of features)
#   y_train: array of categorical labels (dtype object) of shape (number of samples,)
#   n_estimators: number of decision trees, default = 100
# OUTPUT
#   model: Random Forest classifier

    # Create Random Forest with n_estimators and train using X/y_train
    rf_model = ()
    rf_model = RandomForestClassifier(n_estimators=n_estimators)
    rf_model.fit(X_train, y_train)

    return rf_model


def classifier_LSTM(X_train, y_train,
                    X_test, y_test,
                    label_encoder = [],
                    epochs = 20,
                    batch_size = 32,
                    #validation_split = [],
                    model_name= "Default"):
# Classifier Model: Random Forest
# INPUT:
#   X_train: array of floats of shape (number of samples, number of features)
#   y_train: array of categorical labels (dtype object) of shape (number of samples,)
#   label_encoder: label encoder trained on y_train prior to training any models

# OUTPUT
#   model: LSTM Classifier


    # Reformat X train data
    X_train_lstm = np.array(X_train)
    X_test_lstm = np.array(X_test)

    # Get correct number of features
    num_features = X_train_lstm.shape[1]  # Make sure this is the expected number

    # Reshape Data for LSTM Classifier (samples, time_steps, features)
    X_train_lstm = X_train_lstm.reshape((-1, 1, num_features))
    X_test_lstm = X_test_lstm.reshape((-1, 1, num_features))

    # convert y_train to numerical values
    y_train_encoded = label_encoder.transform(y_train)
    y_test_encoded = label_encoder.transform(y_test)

    # Define the inout dimensions of the model
    n_outputs = len(label_encoder.classes_)

    # Build LSTM Model
    model = Sequential([
        LSTM(64, return_sequences=True, input_shape=(1, num_features)),
        Dropout(0.2),
        LSTM(32, return_sequences=False),
        Dropout(0.2),
        Dense(n_outputs, activation="softmax")  # Multi-class classification
    ])

    # ✅ Display Model Summary
    model.summary()

    # ✅ Compile Model
    model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])

    # ✅ Train Model
    history = model.fit(
        X_train_lstm, y_train_encoded,
        epochs=epochs, batch_size=batch_size,
        validation_data=(X_test_lstm, y_test_encoded),
        #validation_split = validation_split,
        verbose=1
    )

    # ✅ Save the Model
    model.save(f"{model_name}.keras")

    # display learning curves and save
```

```
    display_learning_curves(history, label=model_name)


    return model, history

def classifier_CNN(X_train, y_train,
                   X_test, y_test,
                   label_encoder = [],
                   epochs = 20,
                   batch_size = 32,
                   #validation_split = 0.2,
                   model_name= "Default"):
# Classifier Model: Random Forest
# INPUT:
#   X_train: array of floats of shape (number of samples, number of features)
#   y_train: categorical labels (dtype object) of shape (number of samples, number of classes)

# OUTPUT
#   model: CNN Classifier

    # Convert categorical y_train to one-hot

    y_train_encoded = label_encoder.transform(y_train)
    y_train_onehot = to_categorical(y_train_encoded)

    y_test_encoded = label_encoder.transform(y_test)
    y_test_onehot = to_categorical(y_test_encoded)

    # Define the model
    n_outputs = len(label_encoder.classes_)
    num_features = X_train.shape[1]
    model = []
    model = Sequential([
        Conv1D(filters=64, kernel_size=3, activation='relu', input_shape=(num_features, 1)),
        Conv1D(filters=64, kernel_size=3, activation='relu'),
        Dropout(0.5),
        MaxPooling1D(pool_size=2),
        Flatten(),
        Dense(100, activation='relu'),
        Dense(n_outputs, activation='softmax')
    ])

    # ✅ Display Model Summary
    model.summary()

    # Compile the model
    # output must be one-hot encoded
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])



    # Train Model
    history = model.fit(
        X_train, y_train_onehot,
        epochs=epochs, batch_size=batch_size,
        validation_data=(X_test, y_test_onehot),
        #validation_split = validation_split,
        verbose=1
    )

    # ✅ Save the Model
    model.save(f"{model_name}.keras")

    # display learning curves and save
    display_learning_curves(history, label=model_name)


    return model, history


def display_learning_curves(history, label="Default"):

    # Plot Training Performance

    fig_acc, axes = plt.subplots(1, 2, figsize=(10, 4), dpi=120)

    axes[0].plot(history.history["accuracy"], label="Training")#, color="blue")
    axes[0].plot(history.history["val_accuracy"], label="Validation", color="orange", linestyle='dashed')
```

```
    axes[0].set_xlabel("Epochs")
    axes[0].set_ylabel("Accuracy")
    axes[0].set_ylim(0.5,1)
    axes[0].set_title(f"Accuracy Curves: {label}")
    axes[0].legend()


    axes[1].plot(history.history["loss"], label="Training")#, color="blue")
    axes[1].plot(history.history["val_loss"], label="Validation", color="orange", linestyle='dashed')
    axes[1].set_xlabel("Epochs")
    axes[1].set_ylabel("Loss")
    axes[1].set_ylim(0.0,1.0)
    axes[1].set_title(f"Loss Curves: {label}")
    axes[1].legend()
    plt.tight_layout()
    plt.show()

    fig_acc.savefig(f"{label}.png")
```

Personal-Health-Activity-Tracker/TimeSeriesHelperFunctions.py

```python
# Spring 25 AAI-530 Group 5
# Helper Functions for Time Series Forecasting Task
#

# load XYZ data
# INPUT:
#   root_dir: (str) root directory of the dataset. must contain both train and test folders
#   train_test: (str) which folder to load - i.e. "train" or "test"
#   sensor_name: (str) sensor name to be loaded - i.e "body_acc", "body_gryo", "total_acc"
# OUTPUT
#   xyz_data: sequences of time-series samples in x, y, z axes combined. output shape: (number of sequences, 128 timeseries samples, 3)
#   activity_label

import pandas as pd
import keras
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from keras.models import Sequential,load_model
from keras.layers import Dense, Dropout, LSTM, Activation, TimeDistributed, RepeatVector, Input
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, root_mean_squared_error, mean_absolute_error, mean_absolute_percentage_error

import TimeSeries_LossFunctions as ts_loss_fn


def load_sensor_label_data(root_dir='./Dataset', train_test = 'train', sensor_name='body_acc'):
    data_dir = f"{root_dir}/{train_test}"
    #print(f"   Loading {train_test} {sensor_name} data from: {data_dir}")

    # load separate x, y, z sensor readings of 'sensor_name'
    sensor_x = pd.read_csv(f"{data_dir}/Inertial Signals/{sensor_name}_x_{train_test}.txt", sep=r'\s+', header =None).to_numpy()
    sensor_y = pd.read_csv(f"{data_dir}/Inertial Signals/{sensor_name}_y_{train_test}.txt", sep=r'\s+', header =None).to_numpy()
    sensor_z = pd.read_csv(f"{data_dir}/Inertial Signals/{sensor_name}_z_{train_test}.txt", sep=r'\s+', header =None).to_numpy()

    # stack the values
    sensor_xyz = np.stack((sensor_x, sensor_y, sensor_z), axis=2)

    # load the activity labels
    activity_label = pd.read_csv(f"{data_dir}/y_{train_test}.txt", header =None).to_numpy()

    # load the subject labels
    subject_label = pd.read_csv(f"{data_dir}/subject_{train_test}.txt", header =None).to_numpy()

    return sensor_xyz, activity_label, subject_label

# time-series modelling for XYZ sensor data
def ts_forecaster_model1(
    X_train, y_train, # has to be min-max scaled
    sensor_name = 'body_acc',
    user_loss_fn = 'mse',


    epochs = 30,
    batch_size = 500,
    validation_split = 0.2

):
    # Derive variables
    seq_length = X_train.shape[1]
    nb_features_in = X_train.shape[2]
    nb_features_out = y_train.shape[2]
    model_path = f'TimeSeriesPredictor_SENSOR({sensor_name})_LOSS_FN({user_loss_fn}).keras'


    # Define the model

    model = Sequential()
    model = Sequential([
        # Encoder LSTM: encode each ( ,64,3) sample into (, 64, 128) context vectors
        LSTM(128, activation='relu', input_shape=(seq_length, nb_features_in), return_sequences=False),
        Dropout(0.2),

        # Repeat the encoded context vector for each output timestep
        RepeatVector(seq_length),
```

```
        # Decoder LSTM for output sequence
        LSTM(128, activation='relu', return_sequences=True),
        Dropout(0.2),

        # Output layer for predicting x, y, z at each timestep
        TimeDistributed(Dense(nb_features_out, activation= 'linear'))
    ])

    # Compile model along with optimizer
    if(user_loss_fn=='mse'): # default
        model.compile(optimizer='adam', loss=user_loss_fn, metrics=['mae'])
    elif(user_loss_fn=='mase_loss_multi_step'):
        model.compile(optimizer='adam', loss=ts_loss_fn.mase_loss_multi_step, metrics=['mse','mae'])
    elif(user_loss_fn=='trace_mse_loss'):
        model.compile(optimizer='adam', loss=ts_loss_fn.trace_mse_loss, metrics=['mse','mae'])
    else:
        print("ERROR: Invalid user loss function")

    # fit the network
    history = model.fit(X_train, y_train, epochs=epochs, batch_size=batch_size, validation_split=validation_split, verbose=1,
          #callbacks = [keras.callbacks.EarlyStopping(monitor='val_loss', min_delta=0, patience=10, verbose=0, mode='min'),
          #             keras.callbacks.ModelCheckpoint(model_path,monitor='val_loss', save_best_only=True, mode='min', verbose=0)]
          )

    # list all data in history
    print(history.history.keys())




    # Return values
    return model, history, model_path

# evaluate model on test set
def evaluate_XYZ_predictor(
        model, # trained model
        test_sensor_XYZ,
        sensor_scalers, #per-axis MinMaxScaler fitted on Train data
        debug_mode = True
        ):

        # step 1: split the data first into first 64 and last 64
        #X_test, y_test = np.split(test_sensor_XYZ, 2, axis=1)
        X_test = test_sensor_XYZ[:, :64, :]  # First 64 time-series samples
        y_test = test_sensor_XYZ[:, 64:, :]  # Last 64 time-series samples

        #print(f"X_test shape: {X_test.shape}")
        #print(f"y_test shape: {y_test.shape}")

        # Step 3: scale according to train scaler

        X_test_scaled = np.zeros_like(X_test)
        y_test_scaled = np.zeros_like(y_test)


        for axis in range(test_sensor_XYZ.shape[2]):
            #test_XYZ_scaled[:, :, axis] = sensor_scalers[axis].transform(test_sensor_XYZ[:, :,
axis].flatten()).reshape(test_sensor_XYZ.shape[0], test_sensor_XYZ.shape[1])
            X_test_scaled[:,:,axis] = sensor_scalers[axis].transform(X_test[:, :, axis].reshape(-1, 1)).reshape(X_test[:,:,axis].shape)
            y_test_scaled[:,:,axis] = sensor_scalers[axis].transform(y_test[:, :, axis].reshape(-1, 1)).reshape(y_test[:,:,axis].shape)


        #X_test_scaled = sensor_scaler.transform(X_test.reshape(-1, 3)).reshape(X_test.shape)
        #y_test_scaled = sensor_scaler.transform(y_test.reshape(-1, 3)).reshape(y_test.shape)

        # Step 3. Calculate MSE/RMSE/MAE normalized
        scores_test = model.evaluate(X_test_scaled, y_test_scaled, verbose=2)
        mse_scaled = scores_test[0]
        mae_scaled = scores_test[1]
        if(debug_mode):
            print("Metrics on Scaled Test Data:")

            print(f"    MSE (scaled): {mse_scaled:.4f}")
            print(f"    RMSE (scaled): {np.sqrt(mse_scaled):.4f}")
            print(f"    MAE (scaled): {mae_scaled:.4f}")

        # Step 4. Calculate MSE/RMSE/MAE on UNSCALED data
```

Personal-Health-Activity-Tracker/TimeSeries_LossFunctions.py

```python
# custom loss functions for TimeSeries Prediction
# base code generated by Perplexity AI
import keras.ops as K

def mase_loss_multi_step(y_true, y_pred):
    # Assuming y_true and y_pred are of shape (batch_size, 64, features)
    n = K.shape(y_true)[1]  # number of time steps (64)

    # Calculate naive forecast (persistence model)
    # We use the last known value to predict all 64 future values
    #naive_forecast = K.repeat_elements(y_true[:, :1, :], n, axis=1)
    naive_forecast = K.repeat(y_true[:, :1, :], n, axis=1)

    # Calculate mean absolute error of naive forecast
    mae_naive = K.mean(K.abs(y_true - naive_forecast), axis=(1, 2))

    # Calculate mean absolute error of our model's forecast
    mae_forecast = K.mean(K.abs(y_true - y_pred), axis=(1, 2))

    # MASE is the ratio of model's MAE to naive forecast's MAE
    mase = mae_forecast / (mae_naive + 1e-8)  # add a small value to avoid division by zero

    return K.mean(mase)  # Return scalar


def trace_mse_loss(y_true, y_pred):
    # Assuming y_true and y_pred are of shape (batch_size, forecast_horizon, features)
    forecast_horizon = K.shape(y_true)[1]

    def mse_h(h):
        # Calculate MSE for horizon h
        y_true_h = y_true[:, :h, :]
        y_pred_h = y_pred[:, :h, :]
        return K.mean(K.square(y_true_h - y_pred_h))

    # Sum MSE for all horizons
    tmse = K.sum([mse_h(h) for h in range(1, forecast_horizon + 1)])

    return tmse
```

```
        y_pred_scaled = model.predict(X_test_scaled).astype('float32')
        print(f"PRED SHAPE: {y_pred_scaled.shape}")

        y_pred = np.zeros_like(y_pred_scaled)

        for axis in range(test_sensor_XYZ.shape[2]):
            #test_XYZ_scaled[:, :, axis] = sensor_scalers[axis].transform(test_sensor_XYZ[:, :,
axis].flatten()).reshape(test_sensor_XYZ.shape[0], test_sensor_XYZ.shape[1])
            y_pred[:,:,axis] = sensor_scalers[axis].inverse_transform(y_pred_scaled[:, :, axis].reshape(-1,
1)).reshape(y_pred_scaled[:,:,axis].shape)


        #y_pred = sensor_scaler.inverse_transform(y_pred_scaled.reshape(-1, 3)).reshape(y_test.shape)
        y_true = y_test

        # Step 5. Calculate MSE/RMSE/MAE for unscaled data for Interpretability
        # Initialize dictionaries to store results
        metrics = {}

        # Calculate key metric per axis (x, y, z)
        for axis_index, axis_name in enumerate(['x-axis', 'y-axis', 'z-axis']):
                y_true_axis = y_true[:, :, axis_index].reshape(y_true.shape[0], y_true.shape[1], 1)
                y_pred_axis = y_pred[:, :, axis_index].reshape(y_pred.shape[0], y_pred.shape[1], 1)
                y_true_axis_flat = y_true_axis.flatten()
                y_pred_axis_flat = y_pred_axis.flatten()

                mse_axis = mean_squared_error(y_true_axis_flat, y_pred_axis_flat)
                rmse_axis = root_mean_squared_error(y_true_axis_flat, y_pred_axis_flat)
                mae_axis = mean_absolute_error(y_true_axis_flat, y_pred_axis_flat)
                #mase_axis = ts_loss_fn.mase_loss_multi_step(y_true_axis, y_pred_axis).numpy()
                tmse_axis = ts_loss_fn.trace_mse_loss(y_true_axis, y_pred_axis).numpy()

                metrics[axis_name] = {'MSE': mse_axis,
                                      'RMSE': rmse_axis,
                                      'MAE': mae_axis,
                                      #'MASE': mase_axis,
                                      'TraceMSE': tmse_axis}

        # Calculate MSE and MAE across all axes combined
        mse_combined = mean_squared_error(y_true.flatten(), y_pred.flatten())
        rmse_combined = root_mean_squared_error(y_true.flatten(), y_pred.flatten())
        mae_combined = mean_absolute_error(y_true.flatten(), y_pred.flatten())
        #mase_combined = ts_loss_fn.mase_loss_multi_step(y_true, y_pred).numpy()
        tmse_combined = ts_loss_fn.trace_mse_loss(y_true, y_pred).numpy()
        metrics['xyz-Combined'] = {'MSE': mse_combined,
                                   'RMSE': rmse_combined,
                                   'MAE': mae_combined,
                                   #'MASE': mase_combined,
                                   'TraceMSE': tmse_combined}

        # Print metrics if debug mode is True
        if(debug_mode):
            print("Metrics on Unscaled Test Data:")
            for axis_name in metrics:
                    print(f"  {axis_name}:")
                    print(f"    MSE: {metrics[axis_name]['MSE']:.4f}")
                    print(f"    RMSE: {metrics[axis_name]['RMSE']:.4f}")
                    print(f"    MAE: {metrics[axis_name]['MAE']:.4f}")
                    #print(f"    MASE: {metrics[axis_name]['MASE']:.4f}")
                    print(f"    'TraceMSE': {metrics[axis_name]['TraceMSE']:.4f}")

        return metrics, y_pred, y_true, X_test

def time_series_predictor_pipeline(
        root_dir = './Dataset',
        sensor_name = 'body_acc',
        model_name = "ts_model_default",
        user_loss_fn = 'mse',
        num_epochs = 10,
        batch_size = 500,
        val_split = 0.2,
        debug_mode = True,

):
    print(f"TimeSeriesPredictor for SENSOR: {sensor_name} and MODEL: {model_name}")

    # Step 1: Load TRAIN/TEST XYZ data for specified sensor_name
    print(f"TS_STEP 1: Load train xyz data for SENSOR ({sensor_name})")
```

```
    train_sensor_XYZ, train_label, train_person = load_sensor_label_data(root_dir=root_dir, train_test = 'train', sensor_name=sensor_name)
    test_sensor_XYZ, test_label, test_person = load_sensor_label_data(root_dir=root_dir, train_test = 'test', sensor_name=sensor_name)

    # Step 2: Scale training data per 3D axis
    print(f"TS_STEP 2: scale train xyz data per x,y,z axis using MinMaxScaler")
    # initialize MinMaxScaler to range [0, 1]
    #sensor_scaler = MinMaxScaler(feature_range=(0, 1))
    # normalize: first flatten the data per axis, then scale, then reshape to original
    #train_XYZ_scaled = sensor_scaler.fit_transform(train_sensor_XYZ.reshape(-1, 3)).reshape(train_sensor_XYZ.shape)
    #test_XYZ_scaled = sensor_scaler.transform(test_sensor_XYZ.reshape(-1, 3)).reshape(test_sensor_XYZ.shape)

    # Create a list to store scalers for each axis
    sensor_scalers = []

    # Initialize and fit a MinMaxScaler for each axis
    for axis in range(train_sensor_XYZ.shape[2]):
        scaler = MinMaxScaler(feature_range=(0, 1))
        #scaler.fit(X[:, :, axis].reshape(-1, 1))
        scaler.fit(train_sensor_XYZ[:, :, axis].reshape(-1, 1))
        sensor_scalers.append(scaler)

    # Apply the scalers to each axis
    train_XYZ_scaled = np.zeros_like(train_sensor_XYZ)


    for axis in range(train_sensor_XYZ.shape[2]):
        train_XYZ_scaled[:, :, axis] = sensor_scalers[axis].transform(train_sensor_XYZ[:, :, axis].reshape(-1,
1)).reshape(train_sensor_XYZ.shape[0], train_sensor_XYZ.shape[1])
        #test_XYZ_scaled[:, :, axis] = sensor_scalers[axis].transform(test_sensor_XYZ[:, :, axis].reshape(-1,
1)).reshape(test_sensor_XYZ.shape[0], test_sensor_XYZ.shape[1])

    # Step 3: split normalized XYZ data into two, on the time-axis (axis=1):
    print(f"TS_STEP 3: split scaled train xyz data into two along the time-axis (axis=1)")
    #    (1) first 64 time samples : the most recent sensor readings, and
    #    (2) last 64 samples : future sensor samples
    X_train_scaled = train_XYZ_scaled[:, :64, :]  # First 64 time-series samples
    y_train_scaled = train_XYZ_scaled[:, -64:, :]  # Last 64 time-series samples
    #X_test_scaled = test_XYZ_scaled[:, :64, :]  # First 64 time-series samples
    #y_test_scaled = test_XYZ_scaled[:, -64:, :]  # Last 64 time-series samples


    # Step 4: Create TimeSeriesPredictor Model, and train
    print(f"TS_STEP 4: Create, train, and save a TimeSeriesPredictor")
    model, history, model_path = ts_forecaster_model1(
        X_train_scaled, y_train_scaled,
        sensor_name = sensor_name,
        user_loss_fn = user_loss_fn,
        epochs = num_epochs,
        batch_size = batch_size,
        validation_split = val_split)

    # Step 5: Display Loss Curve(s)
    print(f"TS_STEP 5: Display loss curves")
    #fig_acc = plt.figure(figsize=(6, 6))
    #plt.plot(history.history['loss'])
    #plt.plot(history.history['val_loss'])
    #plt.title(f"TimeSeriesPredictor Loss({user_loss_fn}) ({sensor_name})")
    #plt.ylabel('loss')
    #plt.xlabel('epoch')
    #plt.legend(['train', 'val'], loc='upper left')

    # display learning curves and save
    # Save the Model
    model_name = f"TimeSeries Forecaster for Sensor ({sensor_name})"
    tshf_display_learning_curves(history, label=model_name)

#    Step 6: Load TEST XYZ data for model evaluation
    print(f"TS_STEP 6: Load test xyz data for SENSOR {sensor_name}")


    # Step 7: Evaluate
    print(f"TS_STEP 7: Evaluate model on test data")
    metrics_on_test, y_pred, y_true, X_test = evaluate_XYZ_predictor(
        model, # trained model
        test_sensor_XYZ,
        sensor_scalers, #MinMaxScaler fitted on Train data
        debug_mode=debug_mode
        )
```

```
305
306
307
308     #plt.show()
309     #fig_acc.savefig(f"TimeSeriesPredictor Loss ({user_loss_fn})  ({sensor_name}).png")
310
311     return model, history, sensor_scalers, metrics_on_test
312
313
314 def tshf_display_learning_curves(history, label="Default"):
315
316     # Plot Training Performance
317     fig_acc = plt.figure(figsize=(10, 5))
318     plt.plot(history.history["loss"], label="Training")
319     plt.plot(history.history["val_loss"], label="Validation",color="orange", linestyle='dashed')
320     plt.xlabel("Epochs")
321     plt.ylabel("Loss")
322     plt.title(f"Loss Curves: {label}")
323     plt.legend()
324     plt.show()
325
326     fig_acc.savefig(f"{label}.png")
327
328
329
```